

II BOB. Matplotlib yordamida grafiklar chizish (Plotting with Matplotlib)

2.1. Oddiy grafik (Basic graph)

Oddiy grafiklarni yaratish ma'lumotlarni vizuallashtirishdagi umumiy qadamdir. Ushbu vizual tasvirlar ma'lumotlardagi tendensiyalar, qonuniyatlar va bog'liqliklarni tushunishga yordam beradi. Matplotlib Python-dagi eng mashhur grafik chizish kutubxonalaridan biri bo'lib, u bir necha qator kod yordamida yuqori sifatli grafiklarni osongina yaratish imkonini beradi. Ushbu maqolada Matplotlib yordamida asosiy grafiklarni qanday yaratishni ko'rib chiqamiz.

Grafiklar yaratishni boshlashdan oldin Matplotlib-ni o'rnatishimiz kerak. Uni quyidagi buyruq yordamida o'rnatishimiz mumkin: `pip install matplotlib`

Oddiy grafik (Simple Plot) yaratish misolini ko'rib chiqamiz:

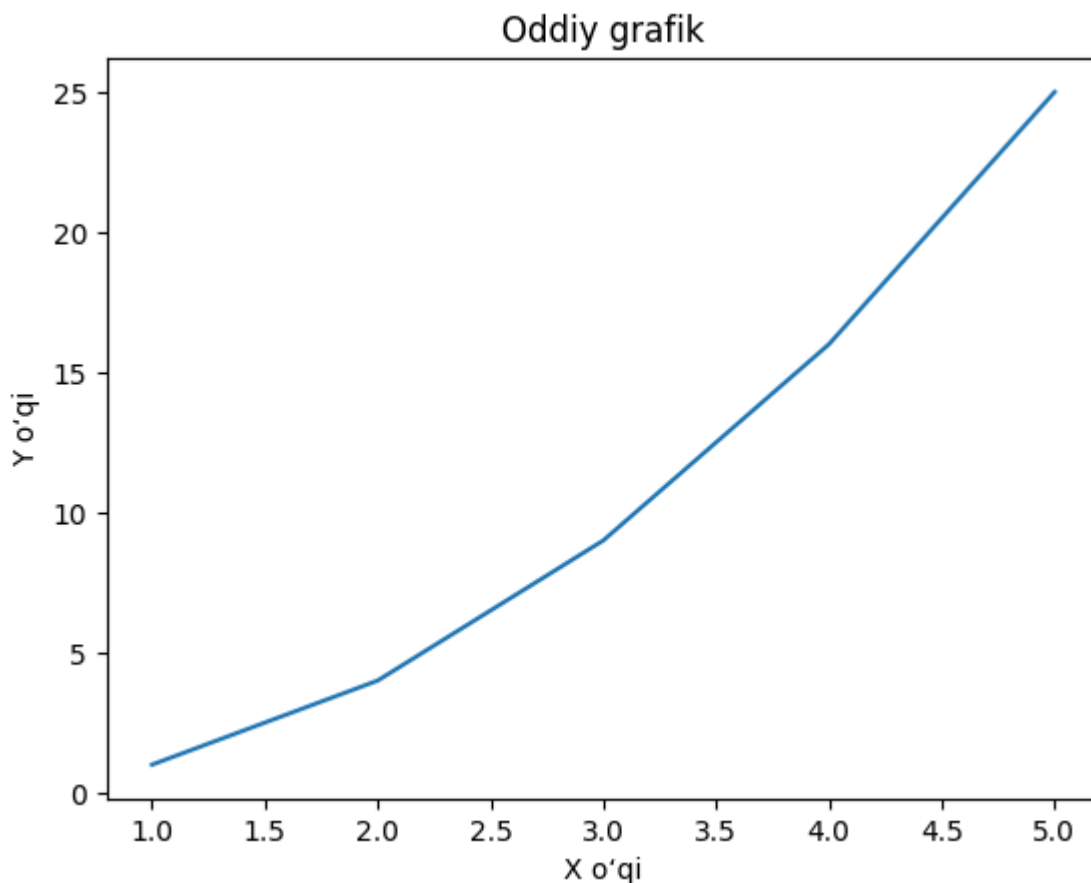
```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]

plt.plot(x, y)

plt.xlabel('X o'qi')
plt.ylabel('Y o'qi')
plt.title('Oddiy grafik')

plt.show()
```



Sintaksis: `plt.plot(x, y, format_string, **kwargs)`

Parametrlar:

- **x:** X o'qi bo'ylab chiziladigan qiymatlar ketma-ketligi.
- **y:** Y o'qi bo'ylab chiziladigan qiymatlar ketma-ketligi.
- **format_string:** Chiziq uslubi, rangi va belgilarini (markers) aniqlaydigan satr (string). Bu ixtiyoriy parametr.
- ****kwargs:** `label` (yorliq), `linewidth` (chiziq qalinligi), `markersize` (belgi o'lchami) kabi boshqa ixtiyoriy parametrlar.

Oddiy grafiklar uchun Matplotlib-ning asosiy funksiyalari

Metod (Funksiya)	Tavsif
<code>plot()</code>	Bu funksiya berilgan ma'lumotlardan grafik yaratadi. U grafikni ekranda ko'rsatmaydi, lekin uni moslashtirish imkonini beradi.
<code>show()</code>	Bu funksiya yaratilgan grafikni ko'rsatadi. Usiz grafik ekranda paydo bo'lmaydi.

Metod (Funksiya)	Tavsif
<code>xlabel()</code> , <code>ylabel()</code>	Bu funksiyalar X va Y o'qlariga nom (yorliq) qo'shadi, bu esa grafikni tushunarliroq qilishga yordam beradi.
<code>title()</code>	Mazmunni (kontekstni) bildirish uchun grafikga sarlavha qo'shadi.
<code>gca()</code>	Bu funksiya joriy o'qlar (Axes) obyektini qaytaradi, bu grafikning ayrim qismlarini (masalan, chekka chiziqlar, shkalalar va h.k.) o'zgartirish uchun foydalidir.
<code>xticks()</code> , <code>yticks()</code>	X va Y o'qlaridagi belgilashlar (shkalalar) qayerda paydo bo'lishini nazorat qiladi. Biz ularning joylashuvi va qadam o'lchamini belgilashimiz mumkin.
<code>legend()</code>	Bitta grafikda bir nechta ma'lumotlar qatori mavjud bo'lganda foydali bo'lgan shartli belgilar izohini (legend) qo'shadi.
<code>annotate()</code>	Asosiy ma'lumot nuqtalarini ta'kidlash uchun grafikning aniq joylariga matn yoki belgilar kabi izohlar qo'shadi.
<code>figure(figsize = (x, y))</code>	Ko'rsatilgan o'lchamdagi shaklni (oynani) yaratadi, bu yerda <code>x</code> — kenglik va <code>y</code> — balandlik.
<code>subplot(r, c, i)</code>	Bitta oyna ichida bir nechta kichik grafiklarni (subplots) yaratadi. <code>r</code> — qatorlar soni, <code>c</code> — ustunlar soni va <code>i</code> — kichik grafikning joylashuv o'rnini bildiradi.
<code>set_xticks</code> , <code>set_yticks</code>	Kichik grafiklarda (subplots) foydali bo'lgan X va Y o'qlaridagi shkalalar diapazoni va qadam o'lchamini moslashtirish imkonini beradi.

Turli funksiyalardan foydalangan holda grafiklarni moslashtirish bo'yicha turli misollarni ko'rib chiqamiz.

1-misol: Bir nechta ma'lumot qatorlariga ega moslashtirilgan chiziqli grafik yaratish

Ushbu misolda biz ikkita turli xil funksiya yordamida chiziqli grafik yaratamiz: $y = x^2$ va $y = 30 - x^2$. Shuningdek, grafikga to'r chiziqlari (gridlines), o'q nomlari, sarlavha va shartli belgilar izohini (legend) qo'shish orqali uni moslashtiramiz.

```
x = [1, 2, 3, 4, 5]
y1 = [1, 4, 9, 16, 25]
y2 = [25, 20, 15, 10, 5]

# Birinchi funksiya (yashil, uzuq-uzuq chiziq, 'o' belgi)
plt.plot(x, y1, label='y = x^2', color='green', linestyle='--', marker='o')

# Ikkinchi funksiya (qizil, tekis chiziq, 'x' belgi)
plt.plot(x, y2, label='y = 30 - x^2', color='red', linestyle='-', marker='x')

# To'r chiziqlarini yoqish
plt.grid(True)

# o'qlarga nom berish
plt.xlabel('X o'qi')
plt.ylabel('Y o'qi')

# Grafik sarlavhasi
plt.title('Bir nechta qatorlarga ega moslashtirilgan chiziqli grafik')

# Shartli belgilar izohi (legend)
plt.legend()

# Grafikni rasm fayli sifatida saqlash
plt.savefig('all_features_plot.png')

# Grafikni ko'rsatish
plt.show()
```



2-misol: Bitta oynada bir nechta kichik grafiklar (Subplots) yaratish

Biz 2x2 o'lchamdagi to'r shaklida to'rtta turli xil kichik grafiklarni (subplots) yaratamiz. Har bir kichik grafik turli xil ma'lumotlar to'plamini ko'rsatadi. Bu usul bir nechta ma'lumotlar to'plamini bitta oynada yonma-yon solishtirmoqchi bo'lganimizda juda foydali.

```
a = [1, 2, 3, 4, 5]
b = [0, 0.6, 0.2, 15, 10, 8, 16, 21]
c = [4, 2, 6, 8, 3, 20, 13, 15]

# 10x10 o'lchamli oyna yaratish
fig = plt.figure(figsize=(10, 10))

# 2x2 to'rdagi 4 ta joylashuv
sub1 = plt.subplot(2, 2, 1)
sub2 = plt.subplot(2, 2, 2)
sub3 = plt.subplot(2, 2, 3)
sub4 = plt.subplot(2, 2, 4)

# 1-grafik (ko'k rang, kvadrat belgilar)
sub1.plot(a, 'sb')
sub1.set_xticks(list(range(0, 10, 1)))
sub1.set_title('1-grafik (1st Rep)')

# 2-grafik (qizil rang, dumaloq belgilar)
sub2.plot(b, 'or')
```

```

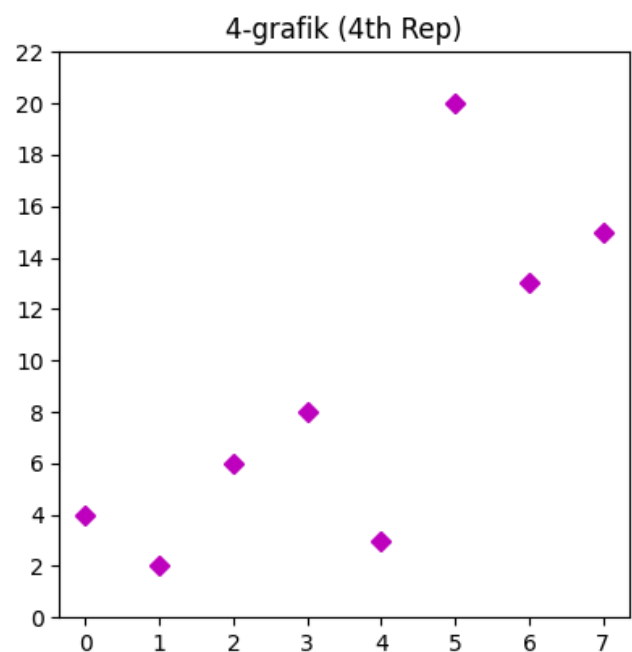
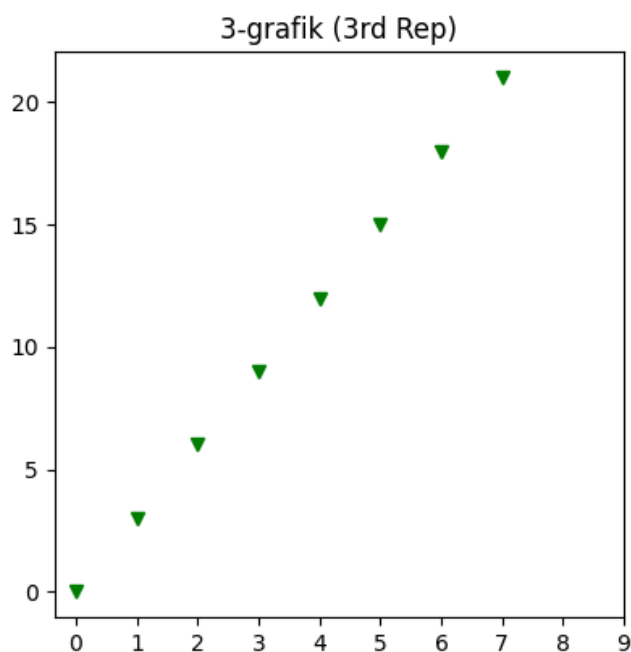
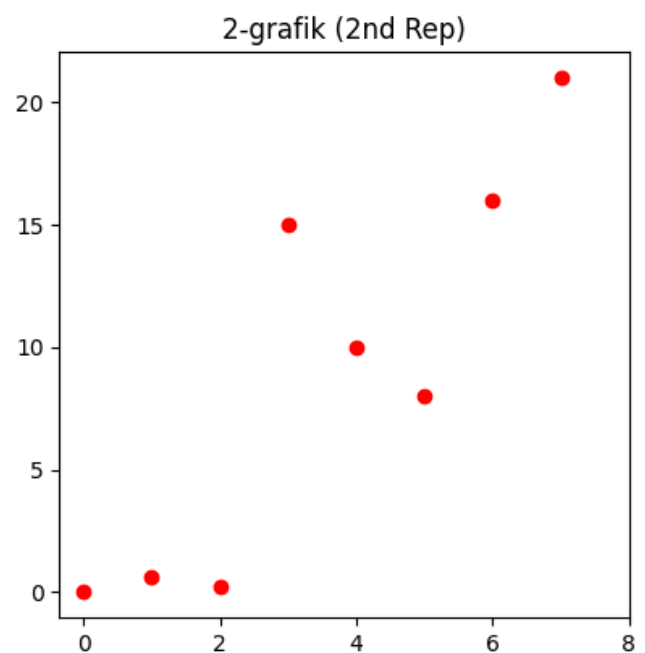
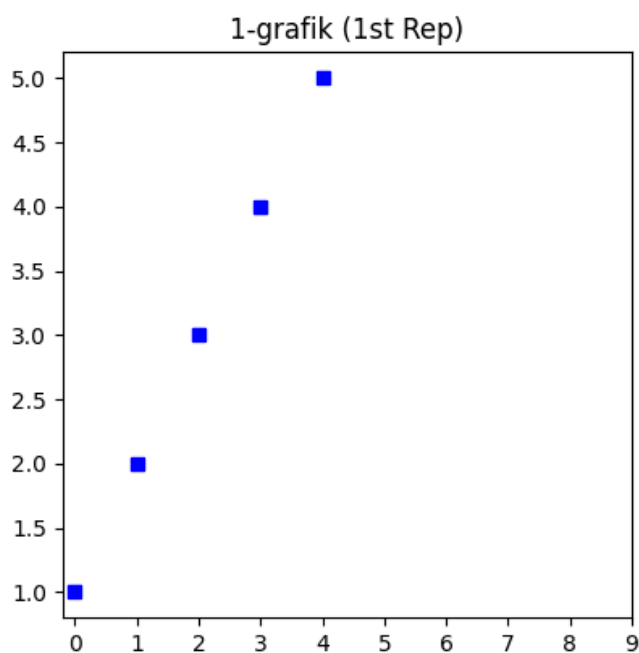
sub2.set_xticks(list(range(0, 10, 2)))
sub2.set_title('2-grafik (2nd Rep)')

# 3-grafik (yashil rang, uchburchak belgilar)
sub3.plot(list(range(0, 22, 3)), 'vg')
sub3.set_xticks(list(range(0, 10, 1)))
sub3.set_title('3-grafik (3rd Rep)')

# 4-grafik (to'q qizil rang, romb belgilar)
sub4.plot(c, 'Dm')
sub4.set_yticks(list(range(0, 24, 2)))
sub4.set_title('4-grafik (4th Rep)')

# Barcha grafiklarni ko'rsatish
plt.show()

```



3-misol: Tarqoq grafik (Scatter Plot) chizish

Tarqoq grafiklar ikki o'zgaruvchi o'rtasidagi bog'liqlikni vizuallashtirish uchun ishlatiladigan asosiy grafik turlaridan biridir. Bu bir o'zgaruvchining boshqasiga qanday bog'liqligini ko'rish yoki korrelyatsiyalarni qidirishda juda foydali.

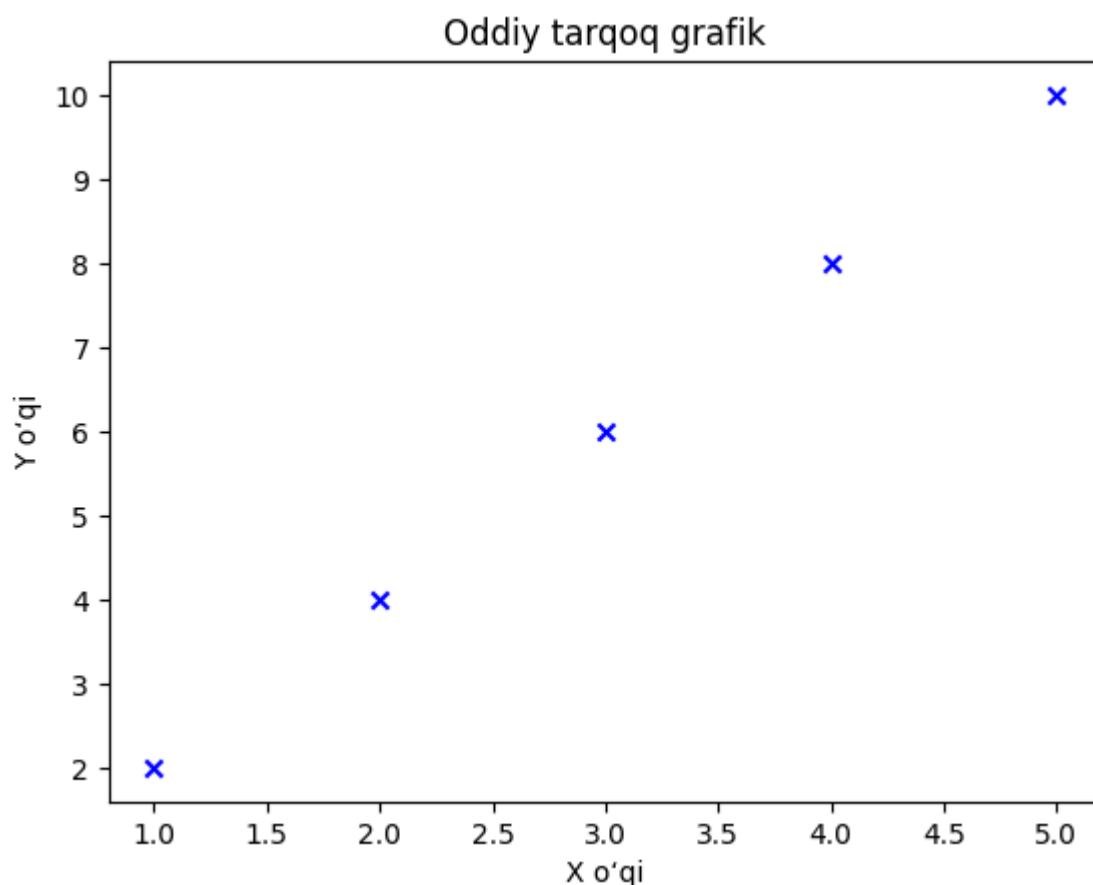
```
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# Tarqoq grafikni chizish (ko'k rang, 'x' belgi)
plt.scatter(x, y, color='blue', marker='x')

# o'qlarga nom berish
plt.xlabel('X o'qi')
plt.ylabel('Y o'qi')

# Grafik sarlavhasi
plt.title('Oddiy tarqoq grafik')

# Grafikni ko'rsatish
plt.show()
```



Ushbu oddiy usullar va misollar yordamida endi biz chiziqli grafiklar, kichik grafiklar (subplots) yoki tarqoq grafiklar bilan ishlashimizdan qat'i nazar, Matplotlib yordamida ma'lumotlarimizni samarali vizuallashtirishni boshlashga tayyormiz.

2.2. Chiziqli grafik (Line Chart)

Chiziqli diagramma yoki chiziqli grafik ma'lumot nuqtalarini to'g'ri chiziq bilan bog'lash orqali ikkita uzluksiz o'zgaruvchi o'rtasidagi bog'liqlikni ko'rsatish uchun foydalaniladigan grafik tasvirdir. Odatda vaqt o'tishi bilan yuz beradigan tendensiyalar, qonuniyatlar yoki o'zgarishlarni vizuallashtirish uchun qo'llaniladi.

Matplotlib-da chiziqli grafiklar ma'lumotlarni chizish uchun sodda va moslashuvchan funksiyalarni taqdim etuvchi `pyplot` qism-kutubxonasi yordamida yaratiladi. Chiziqli grafikda X o'qi odatda erkli o'zgaruvchini, Y o'qi esa erksiz o'zgaruvchini ifodalaydi.

Sintaksis: `plt.plot(x, y, color, linestyle, linewidth, marker)`

Parametrlar:

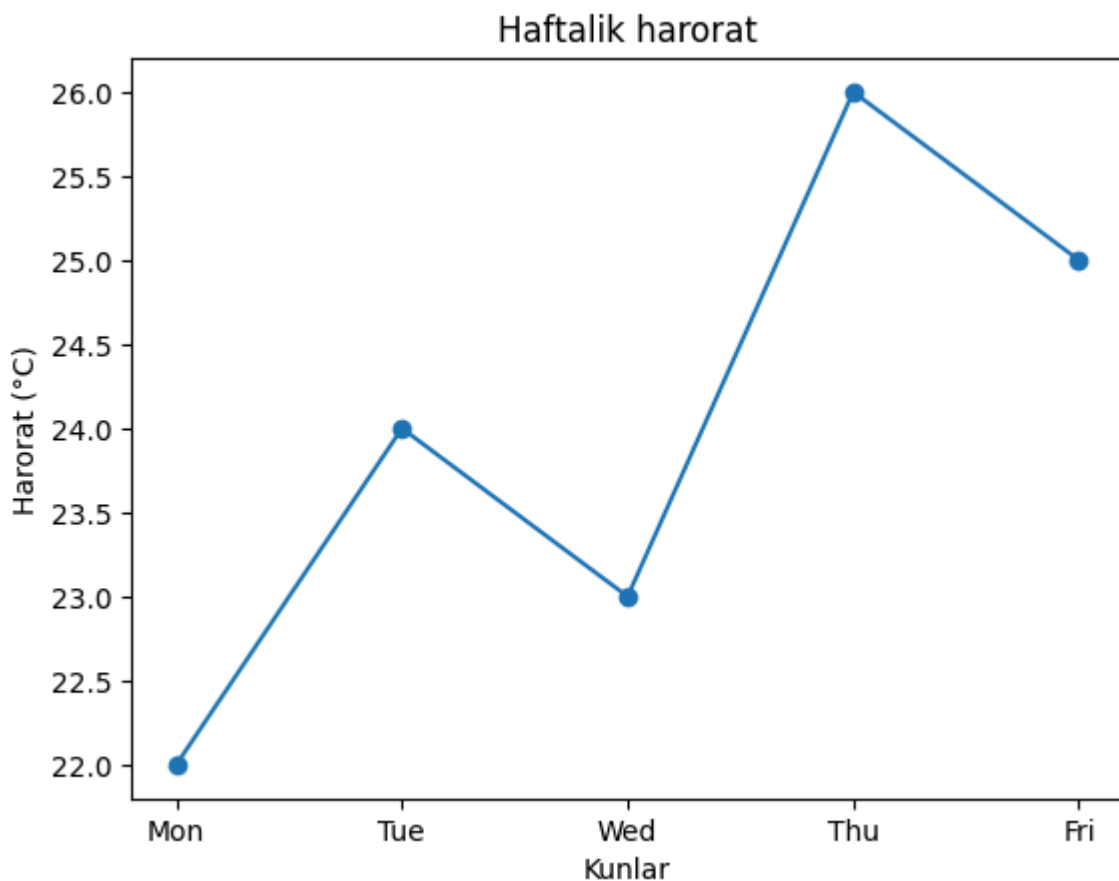
- `x` : X o'qi uchun qiymatlar
- `y` : Y o'qi uchun mos keluvchi qiymatlar
- `color` : Chiziq rangini belgilaydi
- `linestyle` : Chiziq uslubini aniqlaydi
- `linewidth` : Chiziq qalinligini nazorat qiladi
- `marker` : Ma'lumot nuqtalariga belgilar qo'shadi

Matplotlib-da chiziqli grafik yordamida haftalik haroratni vizuallashtiradigan oddiy misolni ko'rib chiqamiz:

```
import matplotlib.pyplot as plt
import numpy as np

days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']
temperature = [22, 24, 23, 26, 25]

plt.plot(days, temperature, marker='o')
plt.title('Haftalik harorat')
plt.xlabel('Kunlar')
plt.ylabel('Harorat (°C)')
plt.show()
```

1. Matplotlib-da oddiy chiziqli grafik (Simple Line Plot)

Chiziqli grafik ikkita uzluksiz o'zgaruvchi o'rtasidagi bog'liqlikni ifodalash uchun ishlatiladi. Matplotlib oddiy va murakkab chiziqli grafiklarni samarali yaratish uchun o'zining `pyplot` moduli orqali `plot()` funksiyasini taqdim etadi.

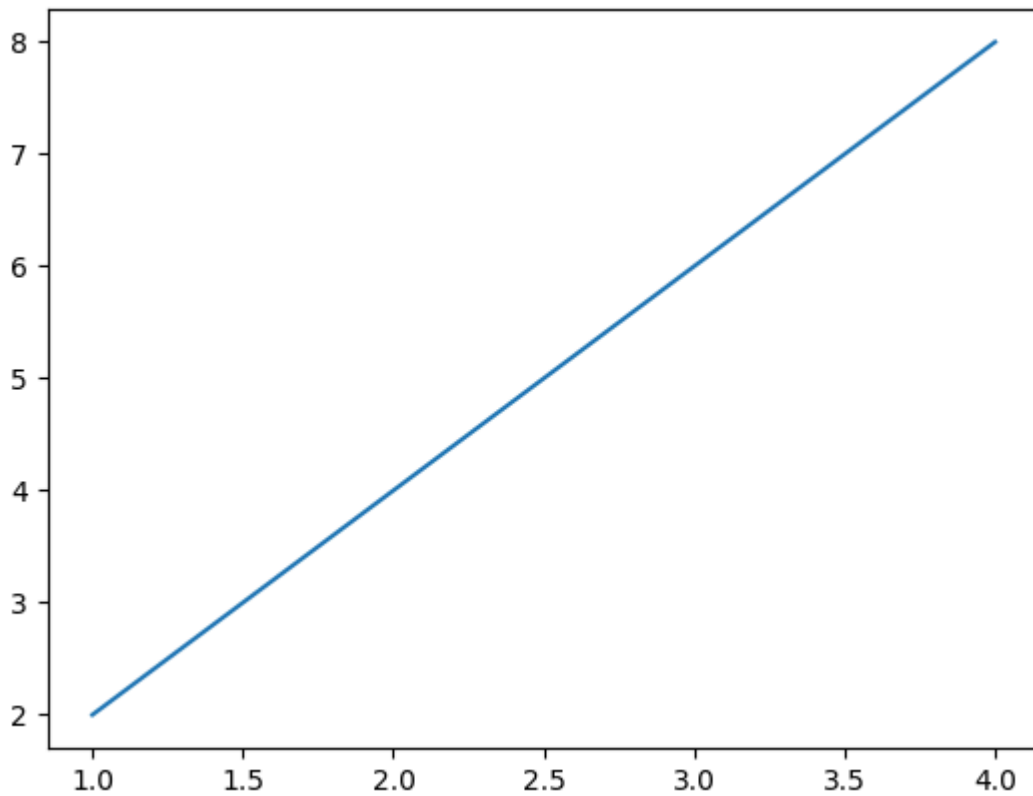
Oddiy chiziqli grafik

Asosiy chiziqli grafikni yaratish uchun `plot()` funksiyasidan foydalanishingiz mumkin. Bu funksiya X va Y o'qlaridagi ma'lumot nuqtalarini tutashtirish orqali chiziq chizadi, bu esa ikkita uzluksiz o'zgaruvchi o'rtasidagi bog'liqlikni vizuallashtirishni osonlashtiradi.

```
import numpy as np

x = np.array([1, 2, 3, 4])
y = x * 2

plt.plot(x, y)
plt.show()
```



Chiziqli grafikga o‘q nomlari va sarlavha qo‘shish

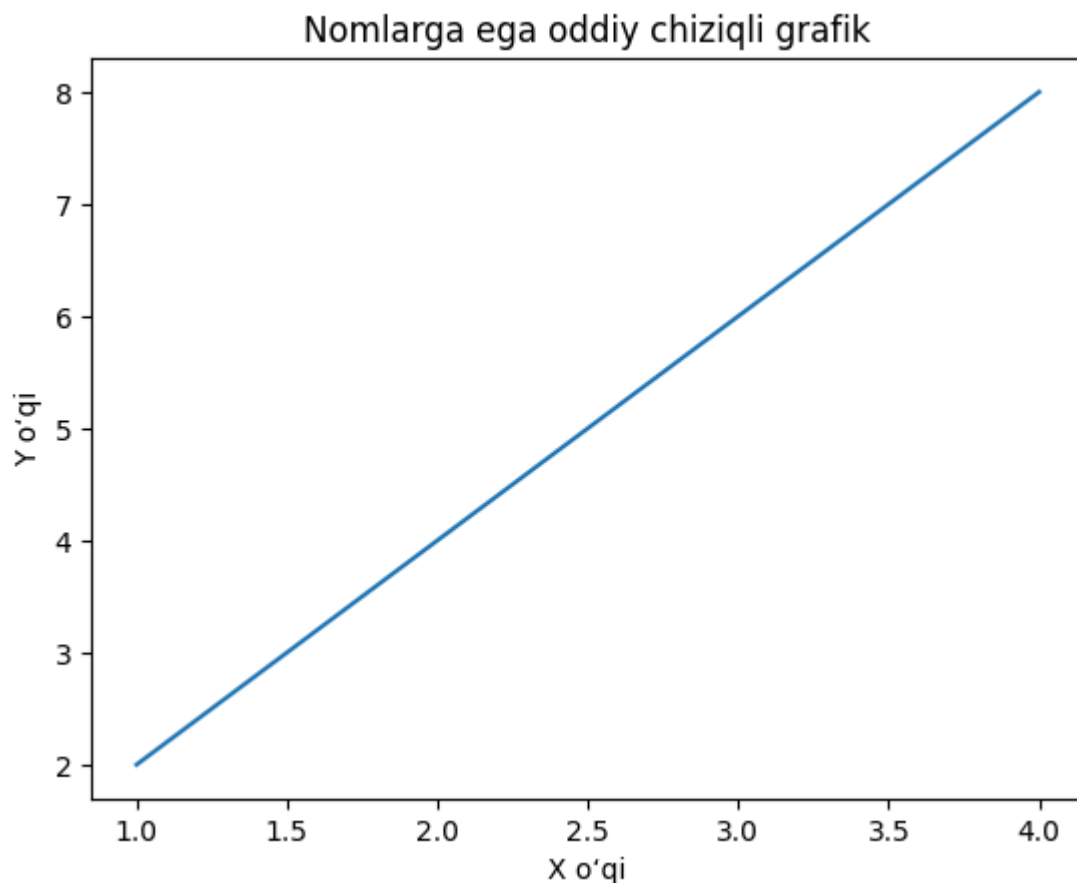
O‘qiluvchanlikni yaxshilash uchun `xlabel()`, `ylabel()` va `title()` funksiyalaridan foydalanishingiz mumkin. Bu funksiyalar o‘qlarni va chiziqli grafikning maqsadini aniq belgilashga yordam beradi.

```
x = np.array([1, 2, 3, 4])
y = x * 2

plt.plot(x, y)

plt.xlabel("X o‘qi")
plt.ylabel("Y o‘qi")
plt.title("Nomlarga ega oddiy chiziqli grafik")

plt.show()
```



Chiziqli grafiklarda belgilardan (markers) foydalanish

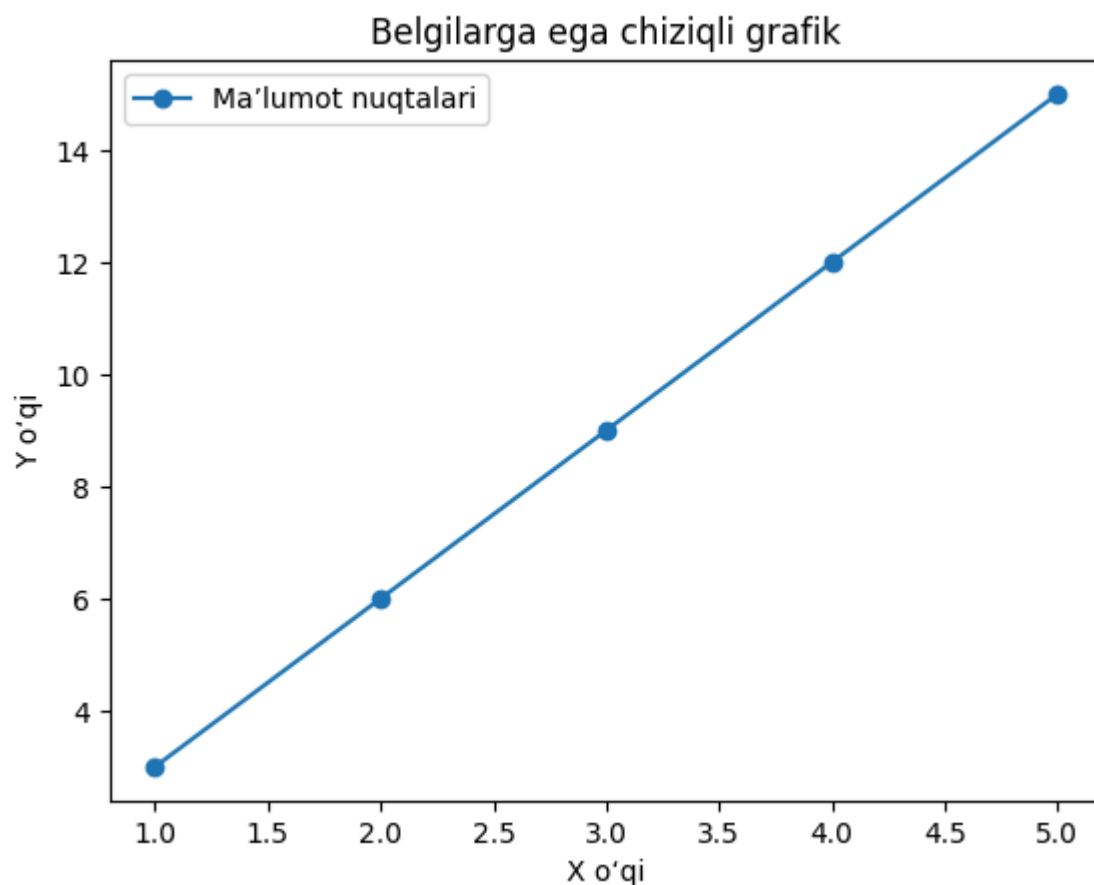
Belgilar chiziqli grafikdagi alohida ma'lumot nuqtalarini ajratib ko'rsatishga yordam beradi va aniq qiymatlarni aniqlashni osonlashtiradi. Ular ayniqsa kichik ma'lumotlar to'plamlari yoki aniq taqqoslashlar uchun juda foydali. `plt.plot()` funksiyasidagi `marker` parametri har bir ma'lumot nuqtasini belgilash uchun ishlatiladigan shaklni (simvolni) aniqlaydi.

```
x = np.array([1, 2, 3, 4, 5])
y = [3, 6, 9, 12, 15]

# marker='o' orqali nuqtalarni dumaloq belgi bilan ajratish
plt.plot(x, y, marker='o', linestyle='-', label='Ma'lumot nuqtalari')

plt.xlabel("X o'qi")
plt.ylabel("Y o'qi")
plt.title("Belgilarga ega chiziqli grafik")
plt.legend()

plt.show()
```



2. Chiziqli grafikga to'r chiziqlarini (Grid) qo'shish

To'r chiziqlari grafik bo'ylab qiymatlarni o'qishni va tendensiyalarni kuzatishni osonlashtiradi. To'rlarni `grid()` funksiyasi yordamida yoqishingiz mumkin.

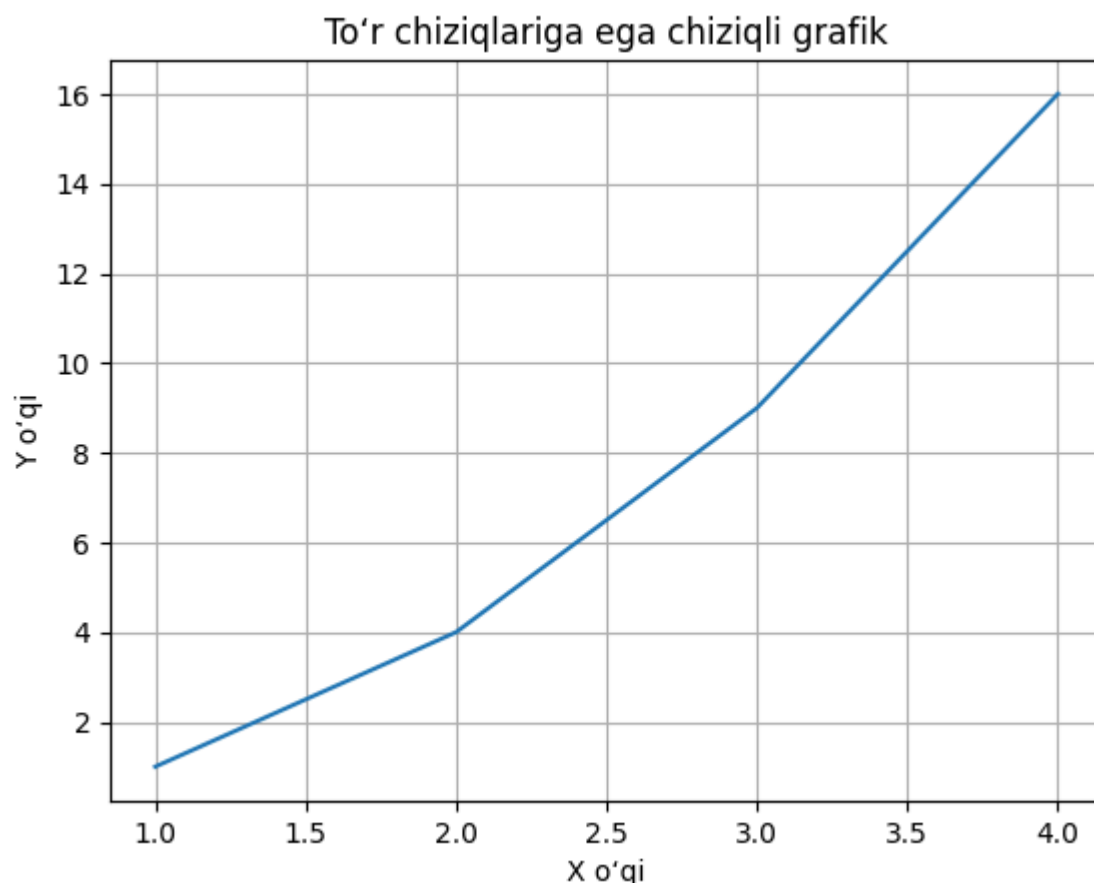
```
x = [1, 2, 3, 4]
y = [1, 4, 9, 16]

plt.plot(x, y)

# To'r chiziqlarini yoqish
plt.grid(True)

plt.xlabel("X o'qi")
plt.ylabel("Y o'qi")
plt.title("To'r chiziqlariga ega chiziqli grafik")

plt.show()
```



3. Izohlarga ega chiziqli grafik (Line Chart with Annotations)

Chiziqli grafikga izohlar qo'shish uchun siz `annotate()` funksiyasidan foydalanishingiz mumkin. Bu funksiya aniqlik va ma'lumotlarni talqin qilishni yaxshilash maqsadida aniq X va Y qiymatlari kabi qo'shimcha ma'lumotlarni bevosita ma'lumot nuqtalarida ko'rsatish imkonini beradi.

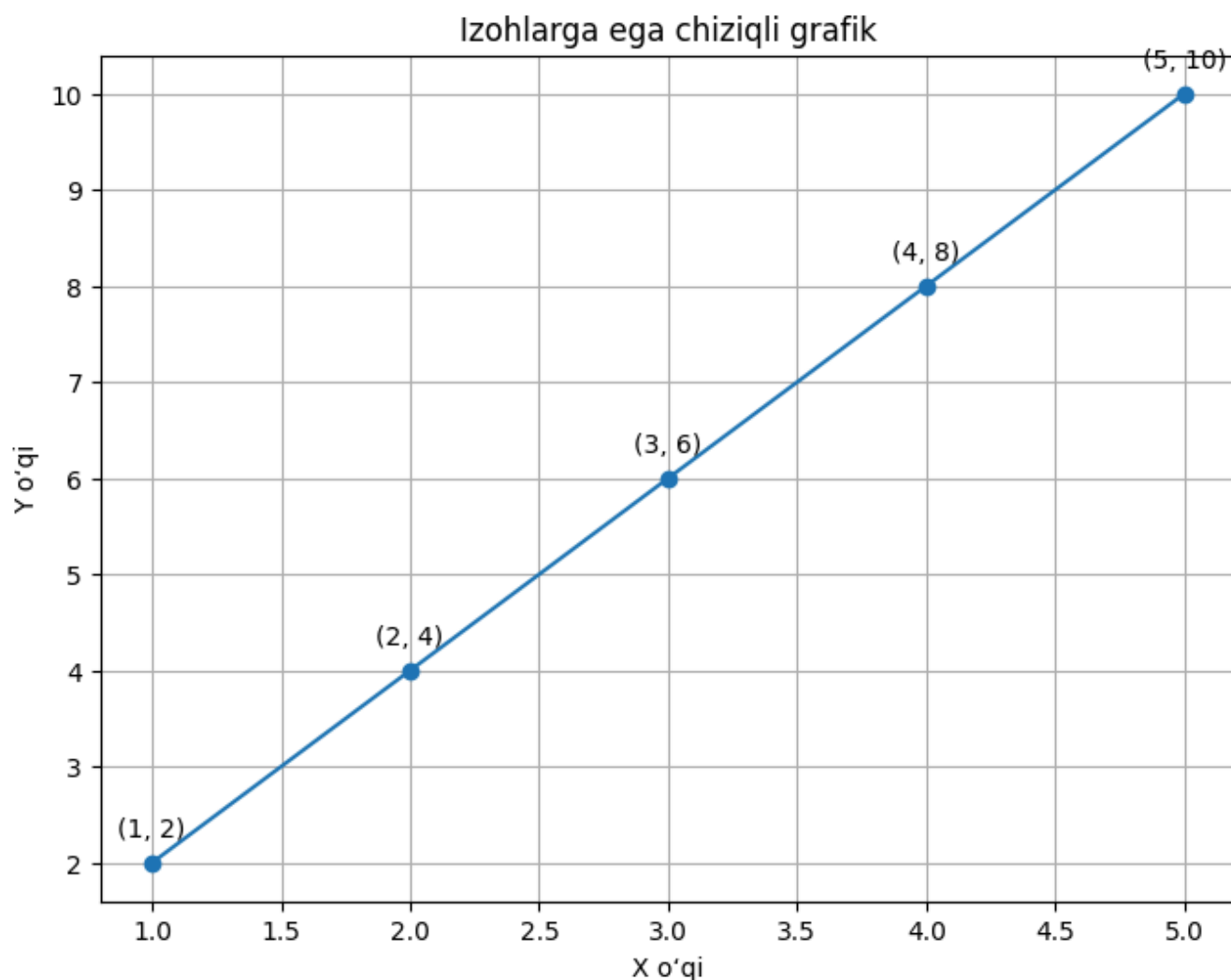
```
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# 8x6 o'lchamli oyna yaratish
plt.figure(figsize=(8, 6))
plt.plot(x, y, marker='o', linestyle='--')

# Har bir nuqtaga izoh (annotatsiya) qo'shish
for xi, yi in zip(x, y):
    plt.annotate(f'({xi}, {yi})',
                 (xi, yi),
                 textcoords="offset points",
                 xytext=(0, 10),
                 ha='center')

plt.title('Izohlarga ega chiziqli grafik')
plt.xlabel('X o'qi')
plt.ylabel('Y o'qi')
```

```
plt.grid(True)
plt.show()
```



4. Matplotlib yordamida bir nechta chiziqli grafiklar (Multiple Line Charts)

Alohida oynalarda bir nechta chiziqli grafiklarni yaratish uchun siz `figure()` funksiyasidan foydalanishingiz mumkin. `figure()` funksiyasini har gal chaqirish yangi chizish maydonini yaratadi, bu esa turli ma'lumotlar to'plamlarini mustaqil ravishda vizuallashtirish va o'zaro solishtirishni osonlashtiradi.

```
x = np.array([1, 2, 3, 4])
y = x * 2

# Birinchi grafikni chizish
plt.plot(x, y)
plt.title("Birinchi chiziqli grafik")
plt.show()

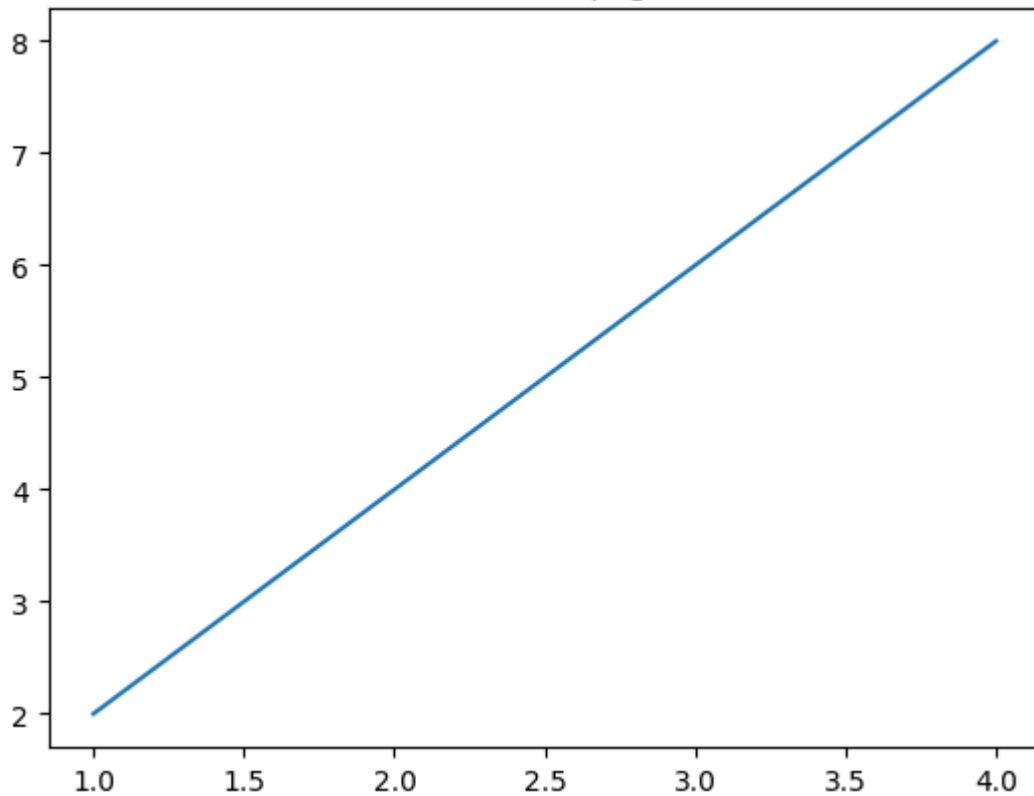
# Yangi oyna yaratish
plt.figure()

x1 = [2, 4, 6, 8]
```

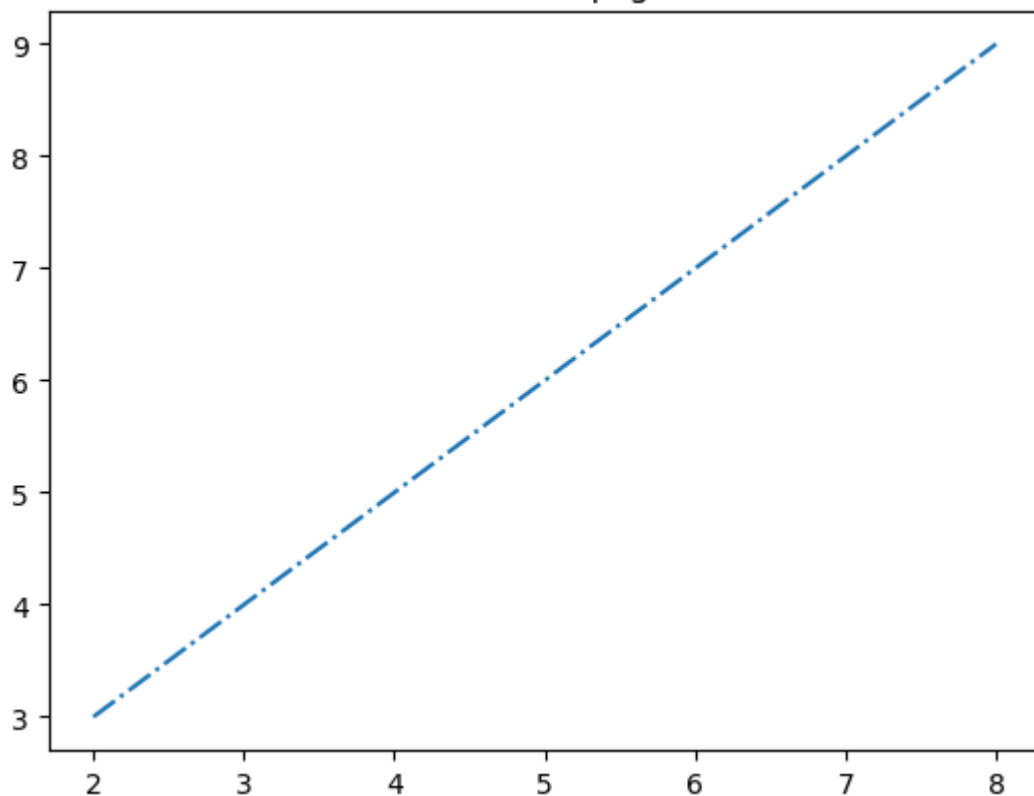
```
y1 = [3, 5, 7, 9]
```

```
# Ikkinchi grafikni chizish ('-.' uzuq-nuqtali chiziq)  
plt.plot(x1, y1, '-.')  
plt.title("Ikkinchi chiziqli grafik")  
plt.show()
```

Birinchi chiziqli grafik



Ikkinchi chiziqli grafik



5. Bitta o'q (Axis) ustida bir nechta grafiklar chizish

Bitta o'q ustida bir nechta chiziqlarni chizish uchun grafikni ko'rsatishdan oldin `plot()` funksiyasini bir necha marta chaqirishingiz mumkin. Bu usul turli ma'lumotlar to'plamlarini bitta koordinatalar tizimida o'zaro solishtirish uchun foydalidir.

```
x = np.array([1, 2, 3, 4])
y = x * 2

x1 = [2, 4, 6, 8]
y1 = [3, 5, 7, 9]

# Birinchi chiziqni chizish
plt.plot(x, y, label='y = 2x')

# Ikkinchi chiziqni chizish
plt.plot(x1, y1, '-.', label='Ikkinchi chiziq')

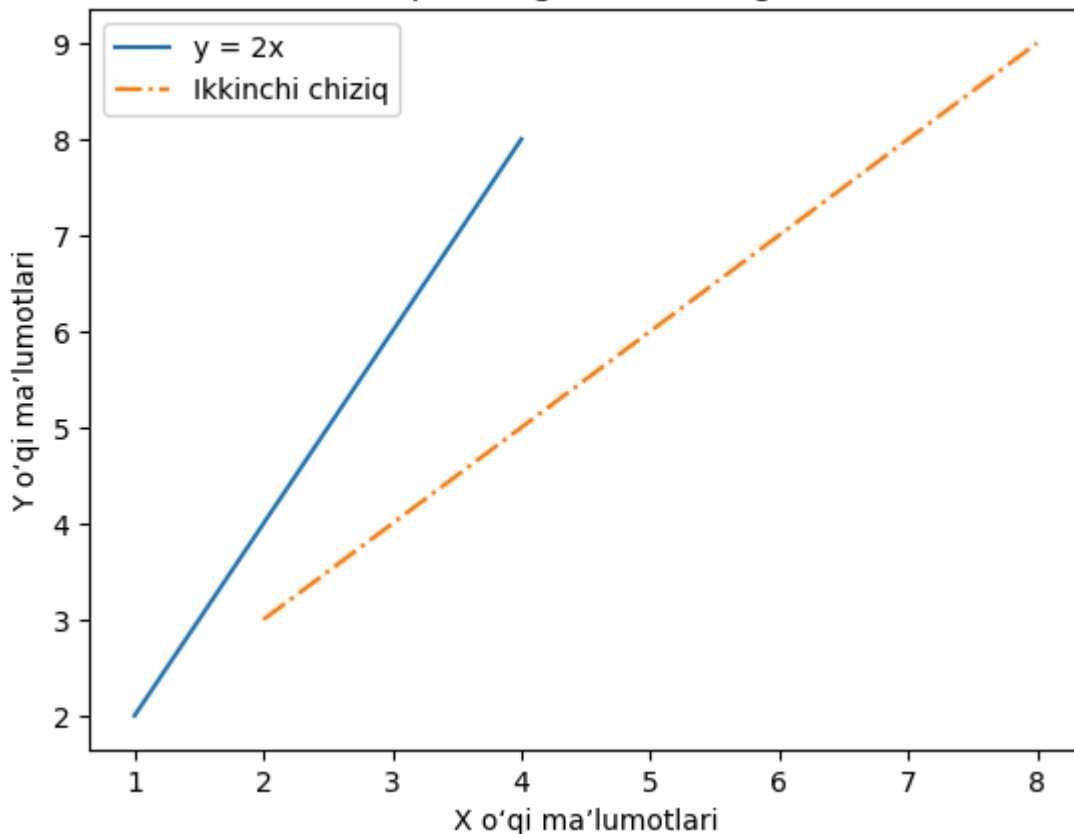
# o'qlarga nom berish
plt.xlabel("X o'qi ma'lumotlari")
plt.ylabel("Y o'qi ma'lumotlari")

# Grafik sarlavhasi
plt.title("Bitta o'q ustidagi bir nechta grafiklar")

# Shartli belgilar izohi
plt.legend()

# Grafikni ko'rsatish
plt.show()
```


Bitta o'q ustidagi bir nechta grafiklar



6. Ikkita chiziq orasidagi maydonni bo'yash

Ikkita chiziqli grafik orasidagi hududni bo'yash uchun `fill_between()` funksiyasidan foydalanishingiz mumkin. Bu funksiya ikkita egri chiziq orasidagi maydonga soya beradi, bu esa ma'lumotlar to'plamlari o'rtasidagi farq yoki oraliqni vizuallashtirishga yordam beradi.

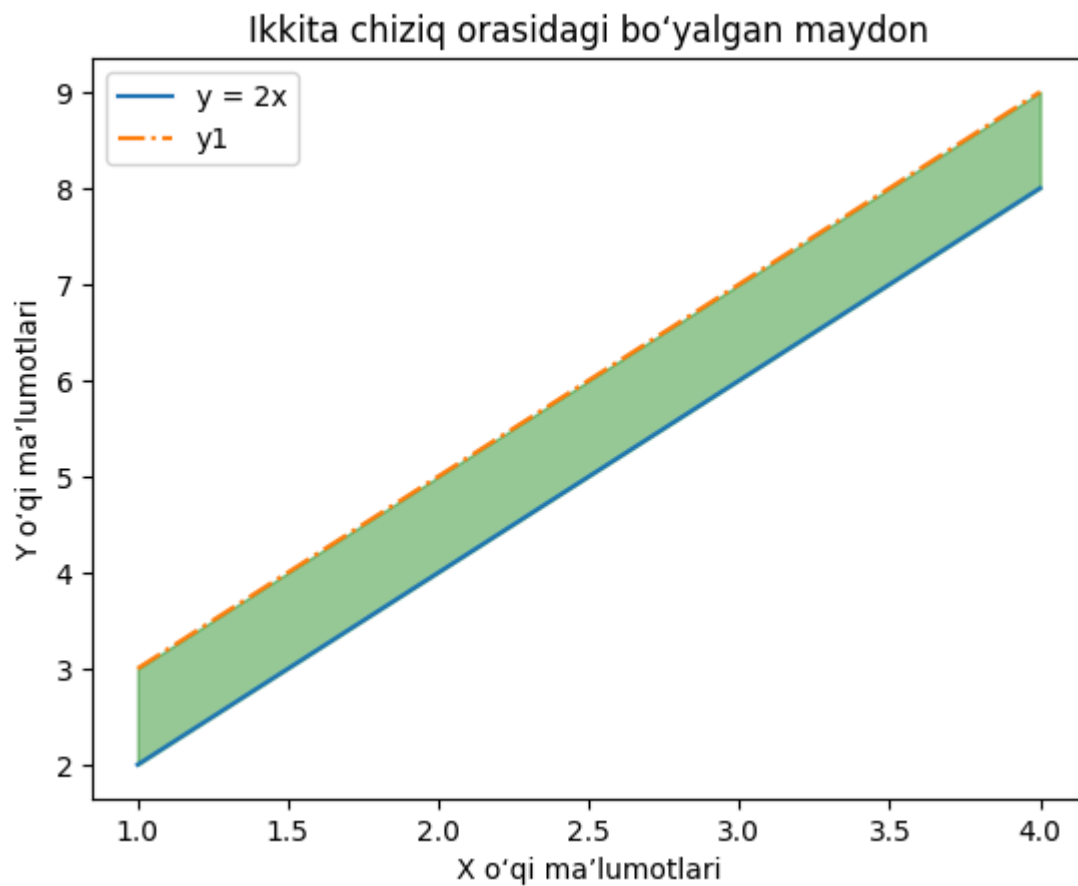
```
x = np.array([1, 2, 3, 4])
y = x * 2
y1 = [3, 5, 7, 9]

plt.plot(x, y, label='y = 2x')
plt.plot(x, y1, '-.', label='y1')

# Ikkita chiziq orasini yashil rang bilan bo'yash (shaffoflik 0.4)
plt.fill_between(x, y, y1, color='green', alpha=0.4)

plt.xlabel("X o'qi ma'lumotlari")
plt.ylabel("Y o'qi ma'lumotlari")
plt.title("Ikkita chiziq orasidagi bo'yalgan maydon")
plt.legend()

plt.show()
```



7. Chiziqli grafikni faylga saqlash

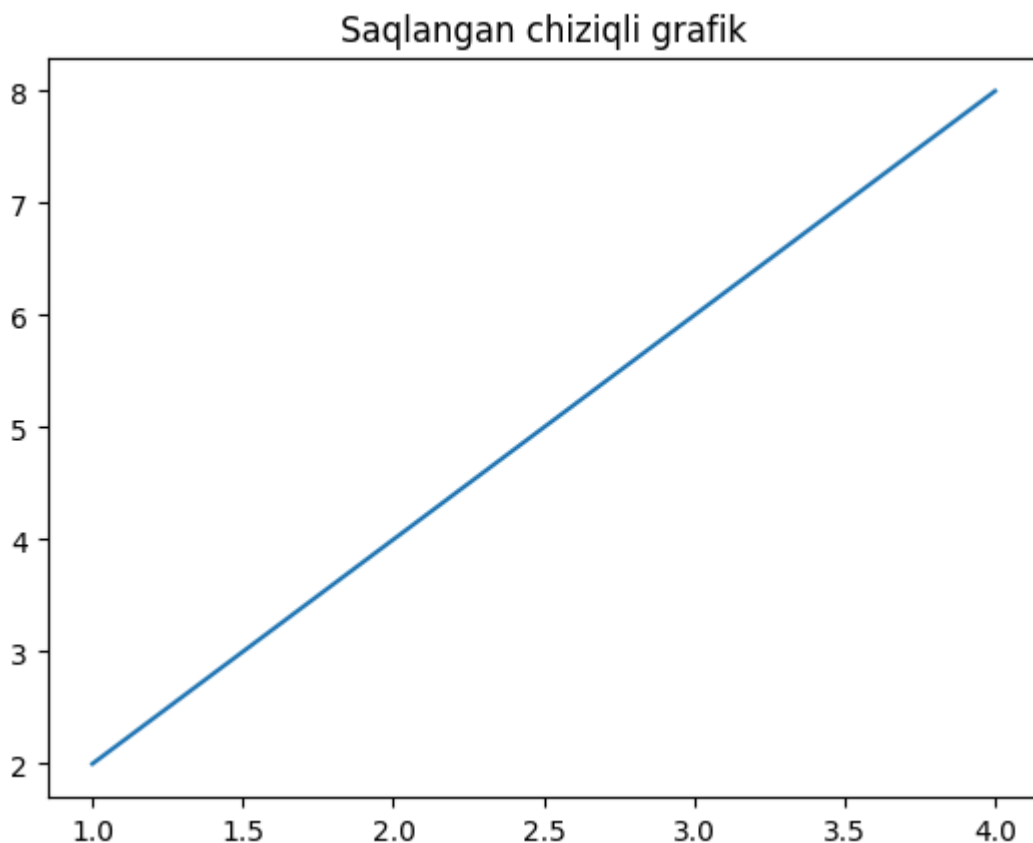
Grafikni ekranda ko'rsatish o'rniga, uni `savefig()` funksiyasi yordamida rasm sifatida saqlashingiz mumkin.

```
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

plt.plot(x, y)
plt.title("Saqlangan chiziqli grafik")

# Grafikni rasm fayli sifatida saqlash
plt.savefig("line_plot.png")

plt.show()
```



8. Trigonometrik funksiyalarni chizish

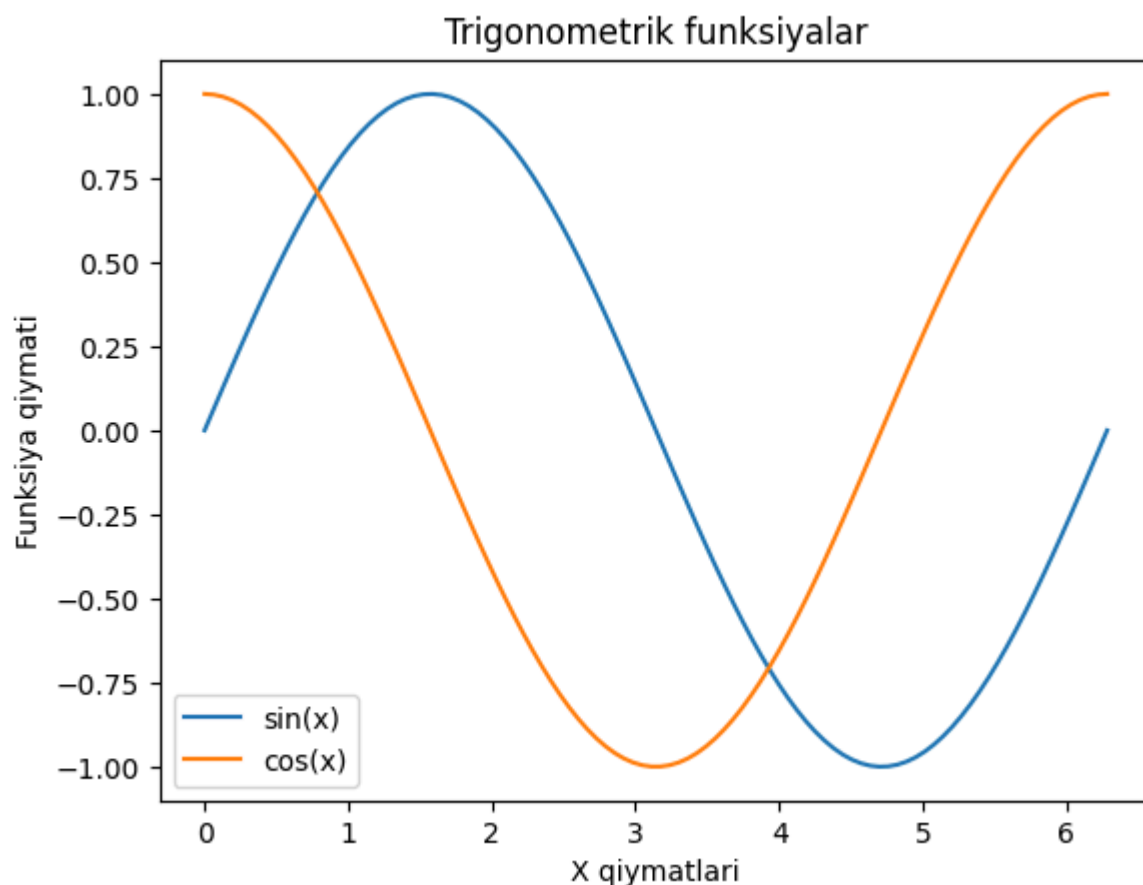
Sinus, kosinus va tangens kabi trigonometrik funksiyalarni uzluksiz oraliqda kiritish qiymatlarini yaratish orqali `plt.plot()` yordamida vizuallashtirish mumkin. `numpy.linspace()` funksiyasi tekis taqsimlangan (evenly spaced) qiymatlarni yaratib, silliq va aniq egri chiziqlar chizish imkonini beradi.

```
# 0 dan 2*pi gacha bo'lgan oraliqda 100 ta nuqta yaratish
x = np.linspace(0, 2 * np.pi, 100)
y_sin = np.sin(x)
y_cos = np.cos(x)

# Funksiyalarni chizish
plt.plot(x, y_sin, label="sin(x)")
plt.plot(x, y_cos, label="cos(x)")

# o'qlarga nom berish
plt.xlabel("X qiymatlari")
plt.ylabel("Funksiya qiymati")
plt.title("Trigonometrik funksiyalar")
plt.legend()

# Grafikni ko'rsatish
plt.show()
```



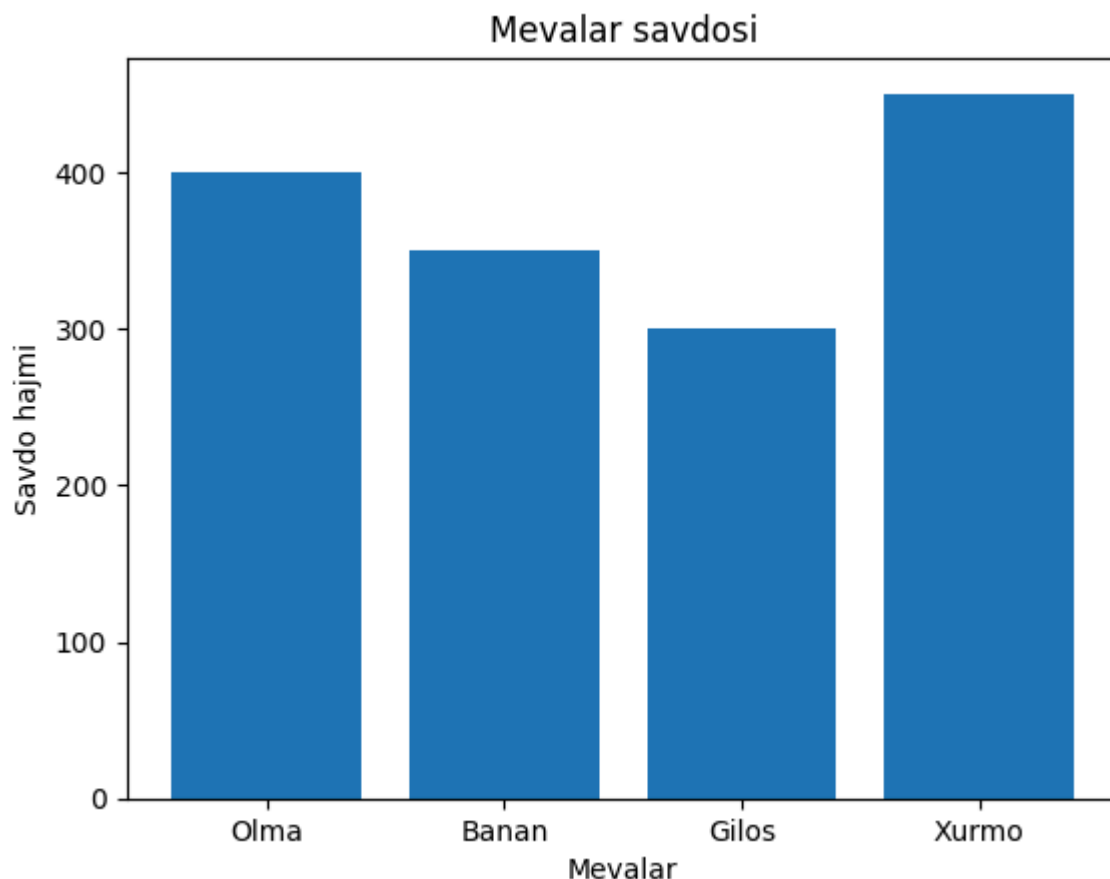
2.3. Ustunli diagramma (Bar Plot)

Ustunli diagramma (Bar Plot)

Ustunli diagramma ma'lumotlar toifalarini ifodalash uchun to'g'ri burchakli ustunlardan foydalanadi, bunda ustunlarning uzunligi yoki balandligi ularning qiymatlariga proporsional (mutanosib) bo'ladi. U diskret (alohida) toifalarni taqqoslaydi, bunda bitta o'q toifalar uchun, ikkinchisi esa qiymatlar uchun ajratiladi.

Turli xil mevalar savdosini vizuallashtiradigan oddiy misolni ko'rib chiqamiz:

```
fruits = ['Olma', 'Banan', 'Gilos', 'Xurmo']  
sales = [400, 350, 300, 450]  
  
# Ustunli diagramma chizish  
plt.bar(fruits, sales)  
  
# Sarlavha va o'qlarga nom berish  
plt.title('Mevalar savdosi')  
plt.xlabel('Mevalar')  
plt.ylabel('Savdo hajmi')  
  
# Grafikni ko'rsatish  
plt.show()
```



Ustunli diagramma (Bar Plot) nima?

Ustunli diagramma (yoki ustunli grafik) turli toifalarni taqqoslash uchun to'g'ri burchakli ustunlardan foydalanadigan vizual ifodalash usulidir. Har bir ustunning balandligi yoki uzunligi u ifodalayotgan qiymatga mos keladi. Odatda X o'qi taqqoslanayotgan toifalarni, Y o'qi esa ushbu toifalarga bog'liq qiymatlarni ko'rsatadi. Ushbu vizual format turli guruhlar bo'ylab miqdorlarni o'zaro solishtirishni juda osonlashtiradi.

`bar()` funksiyasi bir nechta parametrlarni qabul qiladi:

- **x** : Toifalar (masalan, mevalar nomi).
- **height (balandlik)** : Mos keluvchi qiymatlar (masalan, savdo hajmi).
- **width (kenglik)** : Ustunlarning kengligi (standart qiymat — 0.8).
- **bottom (pastki qism)** : Ustunlar uchun asosiy chiziq (standart qiymat — 0).
- **align (tekislash)** : Ustunlarni qanday tekislash kerakligi (`'center'` — o'rta yoki `'edge'` — chetga).

Siz yuborgan kod oldingi ustunli diagramma (bar plot) misolining biroz o'zgartirilgan variantidir. Bu safar `plt.bar()` funksiyasiga `color='violet'` parametri qo'shilgan bo'lib, u barcha ustunlarni binafsha (binafsharang) rangga bo'yaydi.

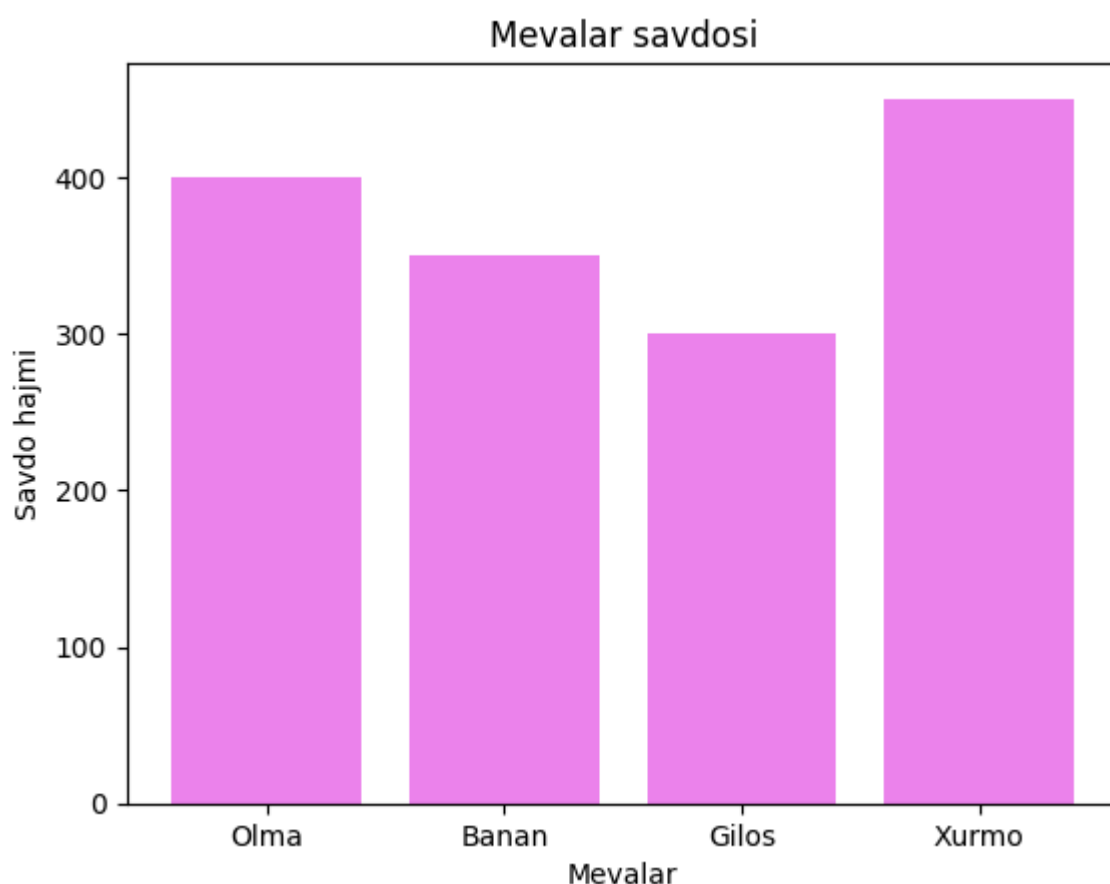
Ushbu kodning tushuntirishi va natijasi quyidagicha:

```
fruits = ['Olma', 'Banan', 'Gilos', 'Xurmo'] # Mevalar nomlari
sales = [400, 350, 300, 450]                # Savdo miqdorlari

# Ustunli diagramma chizish (ustunlar rangi - binafsha)
plt.bar(fruits, sales, color='violet')

# Grafik sarlavhasi va o'q nomlari
plt.title('Mevalar savdosi')
plt.xlabel('Mevalar')
plt.ylabel('Savdo hajmi')

# Grafikni ekranda ko'rsatish
plt.show()
```



Gorizontall ustunli diagrammalar yaratish

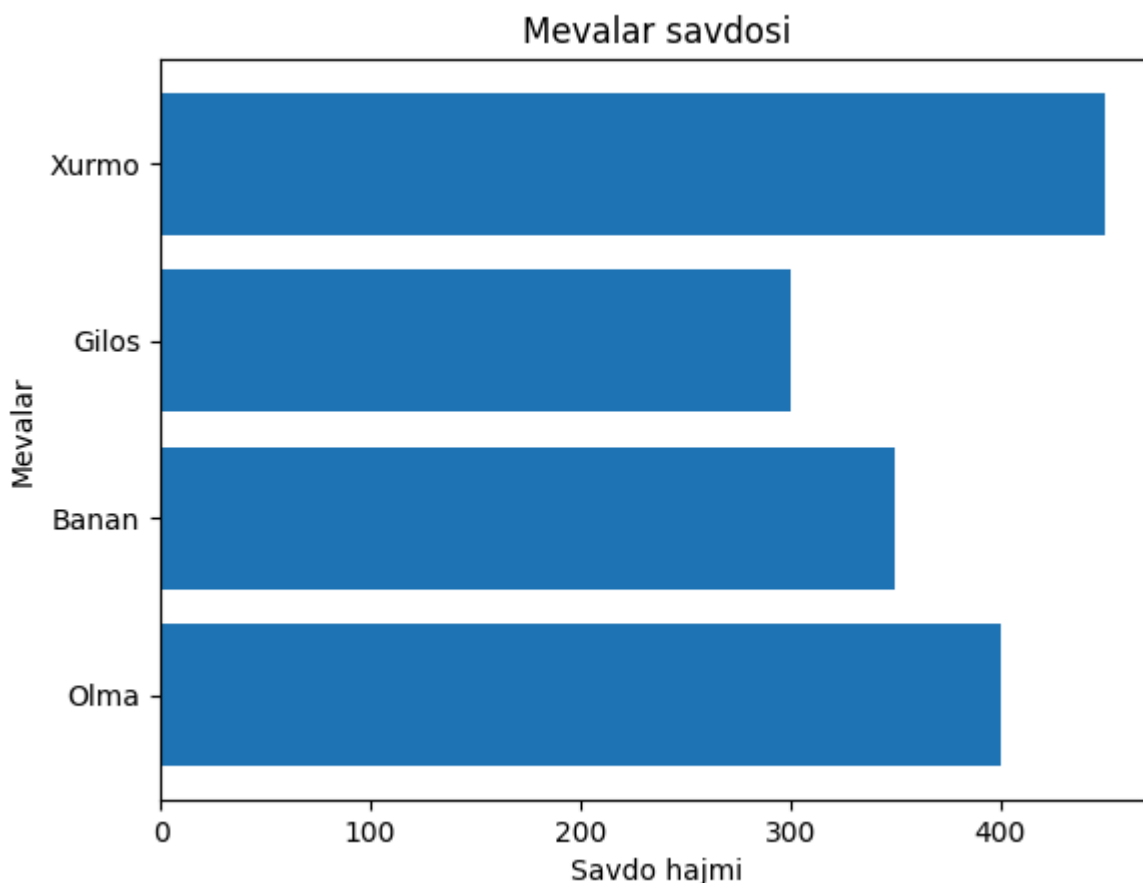
Gorizontall ustunli diagrammalar (Horizontal Bar Plots) yaratish uchun siz `barh()` funksiyasidan foydalanishingiz mumkin. Bu funksiya huddi `bar()` kabi ishlaydi, lekin u ustunlarni yotiq (gorizontall) holatda ko'rsatadi:

(Eslatma: Gorizontall grafikda o'qlar o'rin almashgani sababli, tushunarli bo'lishi uchun kodda X va Y o'qlarining nomlari mos ravishda to'g'rilandi).

```
fruits = ['Olma', 'Banan', 'Gilos', 'Xurmo']
sales = [400, 350, 300, 450]
```

```
# Gorizontal ustunli diagrammani chizish
plt.barh(fruits, sales)

plt.title('Mevalar savdosi')
plt.xlabel('Savdo hajmi') # Gorizontal holatda X o'qi qiymatlarni ko'rsatadi
plt.ylabel('Mevalar')    # Y o'qi esa toifalarni ko'rsatadi
plt.show()
```



Natija (Output): Ekranda toifalar (mevalar) Y o'qida, ularning qiymatlari (savdo hajmi) esa X o'qida joylashgan yotiq ustunli diagramma paydo bo'ladi. Bu usul, ayniqsa, toifalar nomlari uzun bo'lganda o'qishni osonlashtirish uchun juda qulaydir.

Ustun kengligini moslashtirish (Adjusting Bar Width)

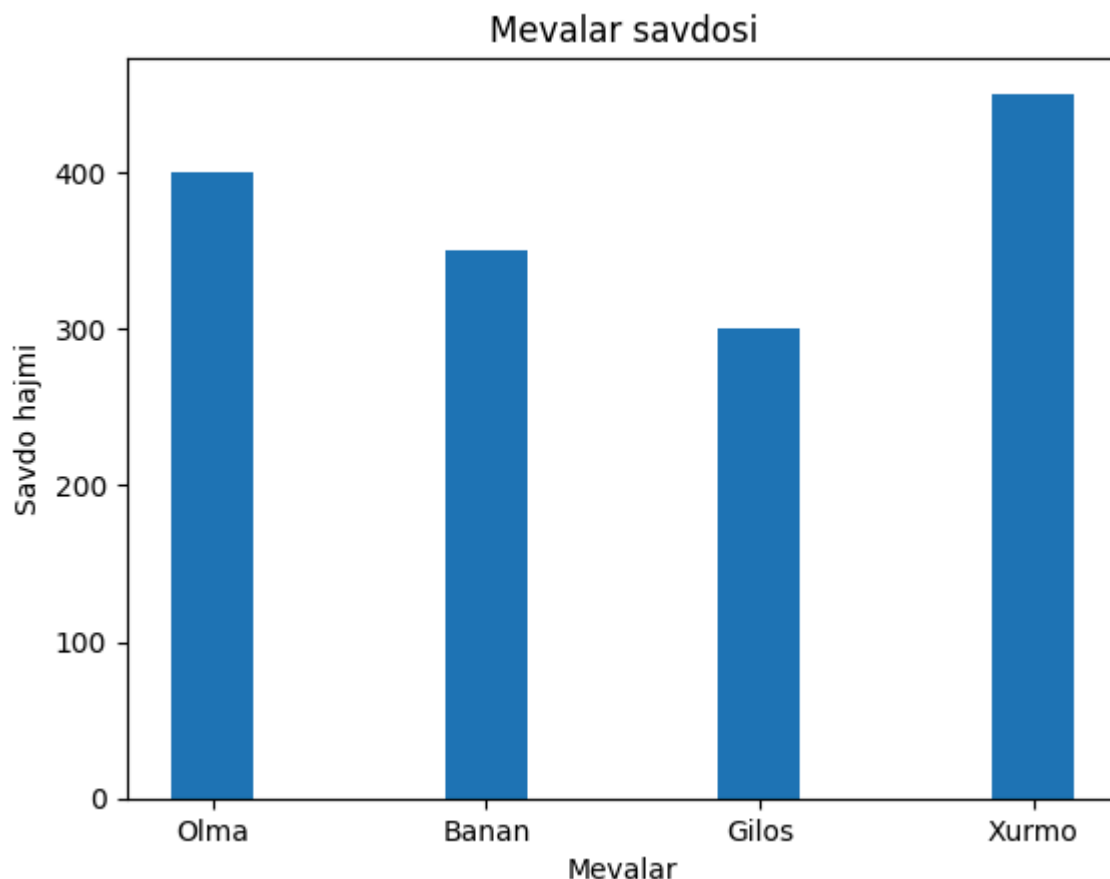
Ustunlarning kengligini `width` parametri yordamida nazorat qilishingiz (o'zgartirishingiz) mumkin:

```
fruits = ['Olma', 'Banan', 'Gilos', 'Xurmo']
sales = [400, 350, 300, 450]

# width=0.3 orqali ustunlar kengligini ingichkalashtirish
plt.bar(fruits, sales, width=0.3)

plt.title('Mevalar savdosi')
```

```
plt.xlabel('Mevalar')
plt.ylabel('Savdo hajmi')
plt.show()
```



Natija (Output): Bu holatda ekranda ustunli diagramma hosil bo'ladi, lekin ustunlar avvalgi misollarga qaraganda ancha **ingichka** bo'ladi. (Eslatma: Matplotlib-da ustunlarning standart kengligi odatda 0.8 ga teng bo'ladi, bu yerda biz uni 0.3 gacha kichraytirdik).

Bir nechta ustunli diagrammalar (Multiple Bar Plots)

Bir nechta ustunli diagrammalar bitta o'zgaruvchi o'zgarganda ma'lumotlar to'plamlari o'rtasida taqqoslashni amalga oshirish uchun ishlatiladi. Biz uni har bir kichik guruh bir-birining ustiga qo'yilgan holda ko'rsatiladigan to'plangan (stacked) ustunli diagrammaga osongina aylantirishimiz mumkin. U ustunlarning qalinligi va X o'qidagi joylashuvini o'zgartirish orqali chiziladi.

Quyidagi ustunli diagramma muhandislik yo'nalishlarida (IT, ECE, CSE) yillar kesimida o'qishni muvaffaqiyatli tamomlagan talabalar sonini ko'rsatadi:

```
# Ustun kengligi
barWidth = 0.25
fig = plt.subplots(figsize=(12, 8))

# Ma'lumotlar to'plami (Har bir yo'nalish bo'yicha talabalar soni)
IT = [12, 30, 1, 8, 22]
```



```

ECE = [28, 6, 16, 5, 10]
CSE = [29, 3, 24, 25, 17]

# X o'qidagi ustunlarning joylashuv o'rnini (koordinatalarini) belgilash
br1 = np.arange(len(IT))
br2 = [x + barWidth for x in br1]
br3 = [x + barWidth for x in br2]

# Ustunlarni chizish
plt.bar(br1, IT, color='r', width=barWidth,
        edgecolor='grey', label='IT')
plt.bar(br2, ECE, color='g', width=barWidth,
        edgecolor='grey', label='ECE')
plt.bar(br3, CSE, color='b', width=barWidth,
        edgecolor='grey', label='CSE')

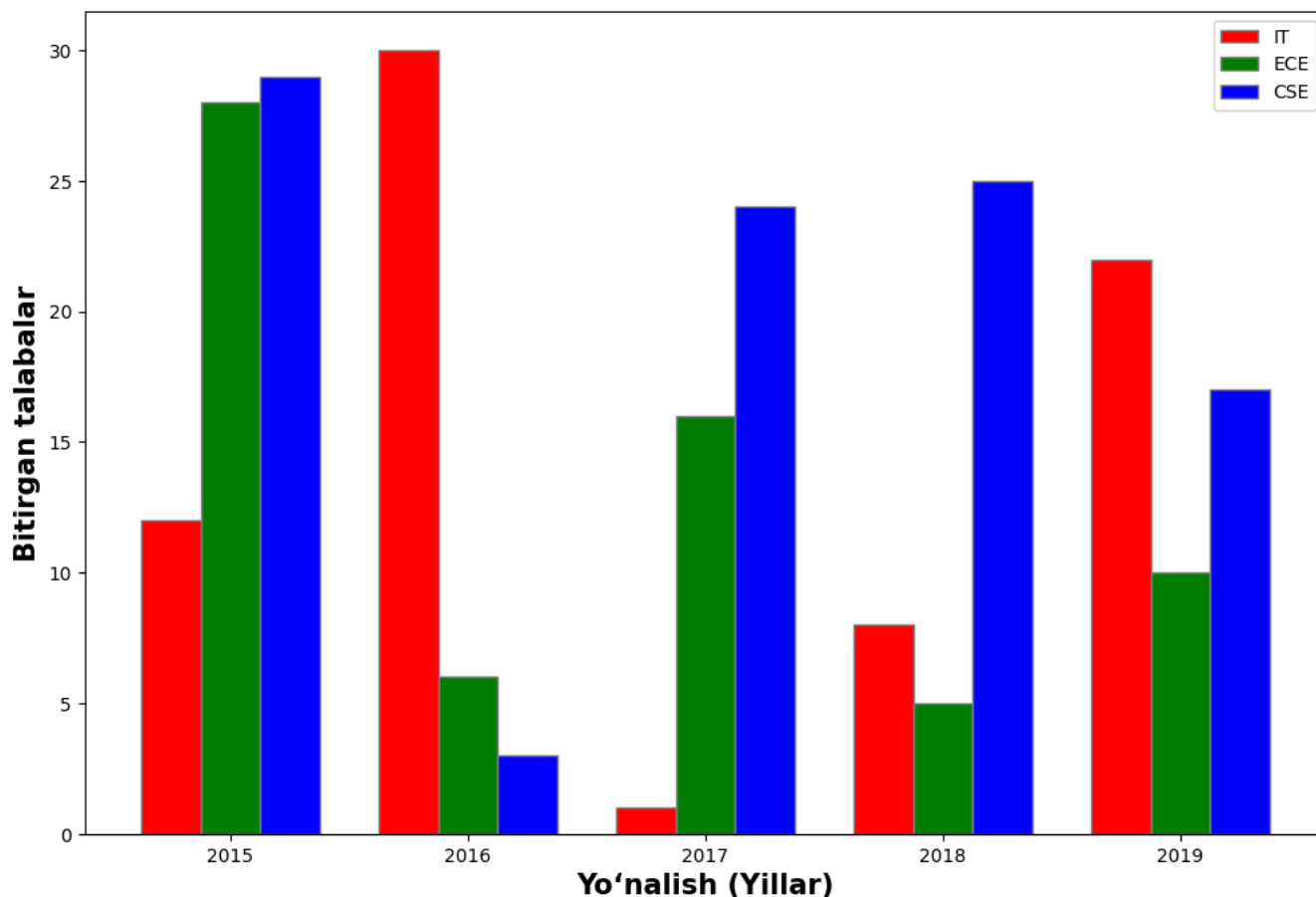
# o'qlarga nom berish
plt.xlabel('Yo'nalish (Yillar)', fontweight='bold', fontsize=15)
plt.ylabel('Bitirgan talabalar', fontweight='bold', fontsize=15)

# X o'qidagi belgilashlarni (yillarni) markazga to'g'rilab yozish
plt.xticks([r + barWidth for r in range(len(IT))],
           ['2015', '2016', '2017', '2018', '2019'])

# Shartli belgilar izohini qo'shish
plt.legend()

# Grafikni ko'rsatish
plt.show()

```



Natija (Output): Ekranda har bir yil uchun uchta turli rangdagi (IT uchun qizil, ECE uchun yashil va CSE uchun ko'k) ustunlar yonma-yon joylashgan guruhlangan diagramma hosil bo'ladi. Bu usul yillar o'tishi bilan turli yo'nalishlardagi bitiruvchilar sonini o'zaro solishtirish uchun juda qulaydir.

Ustma-ust joylashgan ustunli diagramma (Stacked Bar Plot)

Siz yuborgan ushbu kod ustma-ust joylashgan (stacked) ustunli diagramma yaratishni ko'rsatadi. Bu turdagi diagramma bir nechta ma'lumotlar to'plamini (masalan, o'g'il bolalar va qizlar) bitta ustunda ustma-ust joylashtirish orqali umumiy miqdor va uning tarkibiy qismlarini ko'rsatish uchun ishlatiladi. Shuningdek, kodda `yerr` parametri yordamida har bir ustunga ma'lumotlar tarqoqligini ifodalovchi xatolik chiziqlari (error bars - standart og'ish) qo'shilgan.

Kodni tushunarli bo'lishi uchun izohlar va tarjimalar bilan keltirib o'taman:

```
N = 5 # Guruhlar (jamoalar) soni

# Ma'lumotlar to'plami (o'rtacha qiymatlar)
boys = (20, 35, 30, 35, 27)
girls = (25, 32, 34, 20, 25)

# Standart og'ish (xatolik chiziqlari uchun)
boyStd = (2, 3, 4, 1, 2)
girlStd = (3, 5, 2, 3, 3)
```

```

ind = np.arange(N)    # X o'qidagi joylashuvlar (0, 1, 2, 3, 4)
width = 0.35          # Ustunlar kengligi

# 10x7 o'lchamli oyna yaratish
fig, ax = plt.subplots(figsize=(10, 7))

# O'g'il bolalar uchun pastki ustunlarni chizish (yerr - xatolik chizig'i bi
p1 = plt.bar(ind, boys, width, yerr=boyStd)

# Qizlar uchun ustunlarni o'g'il bolalar ustunlarining ustiga chizish
# (bottom=boys parametri orqali ustma-ust joylashtiriladi)
p2 = plt.bar(ind, girls, width,
              bottom=boys, yerr=girlStd)

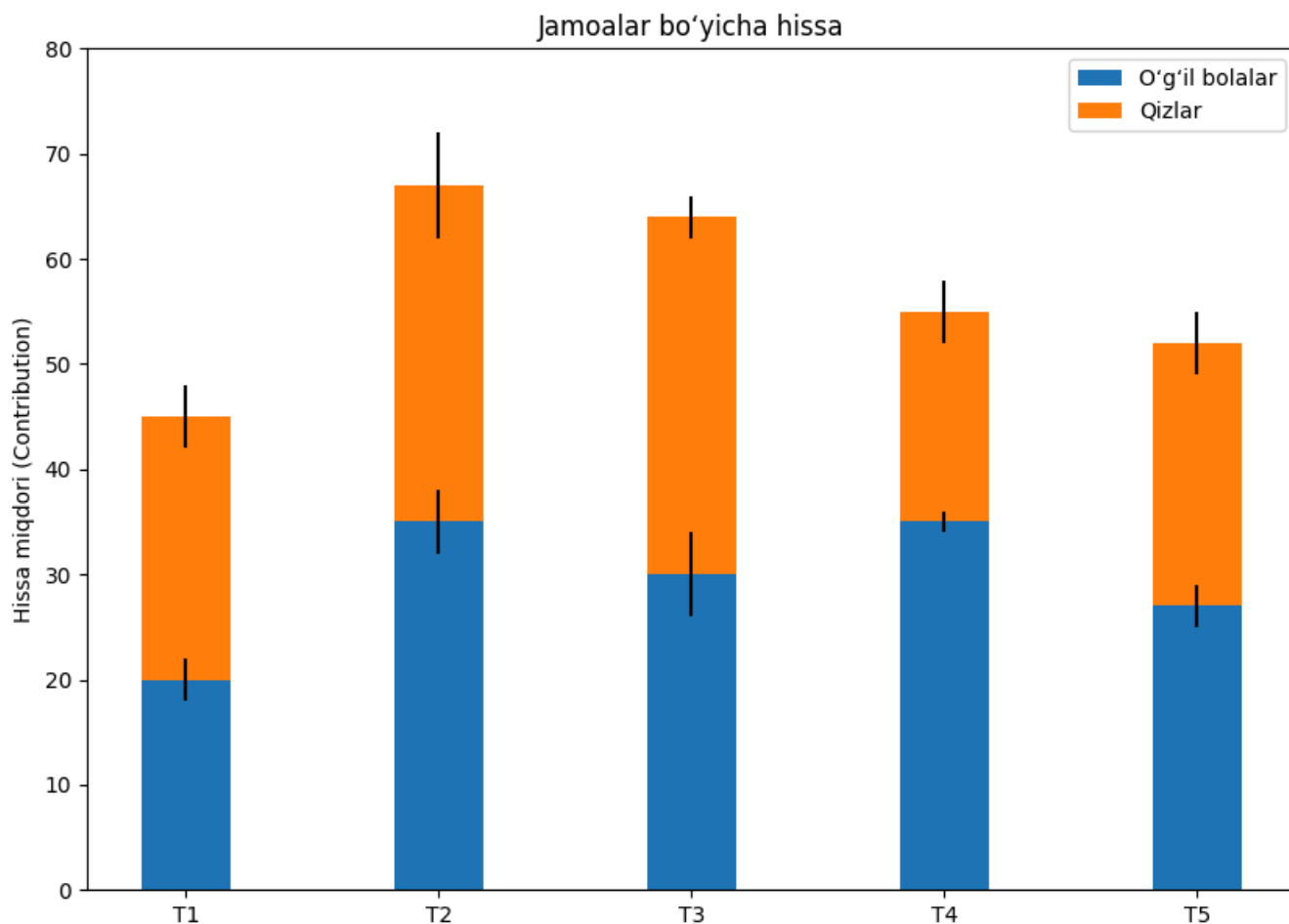
# O'qlarga nom berish va sarlavha qo'shish
plt.ylabel('Hissa miqdori (Contribution)')
plt.title('Jamoalar bo'yicha hissa')

# X va Y o'qlaridagi belgilashlarni sozlash
plt.xticks(ind, ('T1', 'T2', 'T3', 'T4', 'T5'))
plt.yticks(np.arange(0, 81, 10)) # 0 dan 80 gacha 10 qadam bilan

# Shartli belgilar izohi (Legend)
plt.legend((p1[0], p2[0]), ('O'g'il bolalar', 'Qizlar'))

# Grafikni ko'rsatish
plt.show()

```



Natija (Output): Ekranda beshta jamoa (T1 dan T5 gacha) uchun o'g'il bolalar (pastki qism, ko'k rangda) va qizlar (ustki qism, zarg'aldoq rangda) qo'shgan hissalarini ko'rsatuvchi yagona ustunlar qatori hosil bo'ladi. Ustunlarning umumiy balandligi jamoaning umumiy hissasini (o'g'il bolalar + qizlar) bildiradi. Har bir rangli qism markazida ma'lumotlarning standart og'ishini ko'rsatuvchi qora xatolik chiziqlari (I-shaklidagi chiziqlar) ham ko'rsatiladi.

2.4. Nuqtali grafiklar (Scatter Plots)

Nuqtali grafiklar ikkita raqamli o'zgaruvchi o'rtasidagi bog'liqlikni vizuallashtirish uchun eng asosiy vositalardan biridir. `matplotlib.pyplot.scatter()` X va Y koordinatalari bilan belgilangan Dekart tekisligida nuqtalarni chizadi. Har bir nuqta bitta ma'lumot (kuzatuv) qiymatini ifodalaydi, bu esa ikkita o'zgaruvchining o'zaro qanday bog'langanligini (korrelyatsiya), qanday to'planganligini (klaster) yoki taqsimlanganligini vizual tahlil qilish imkonini beradi.

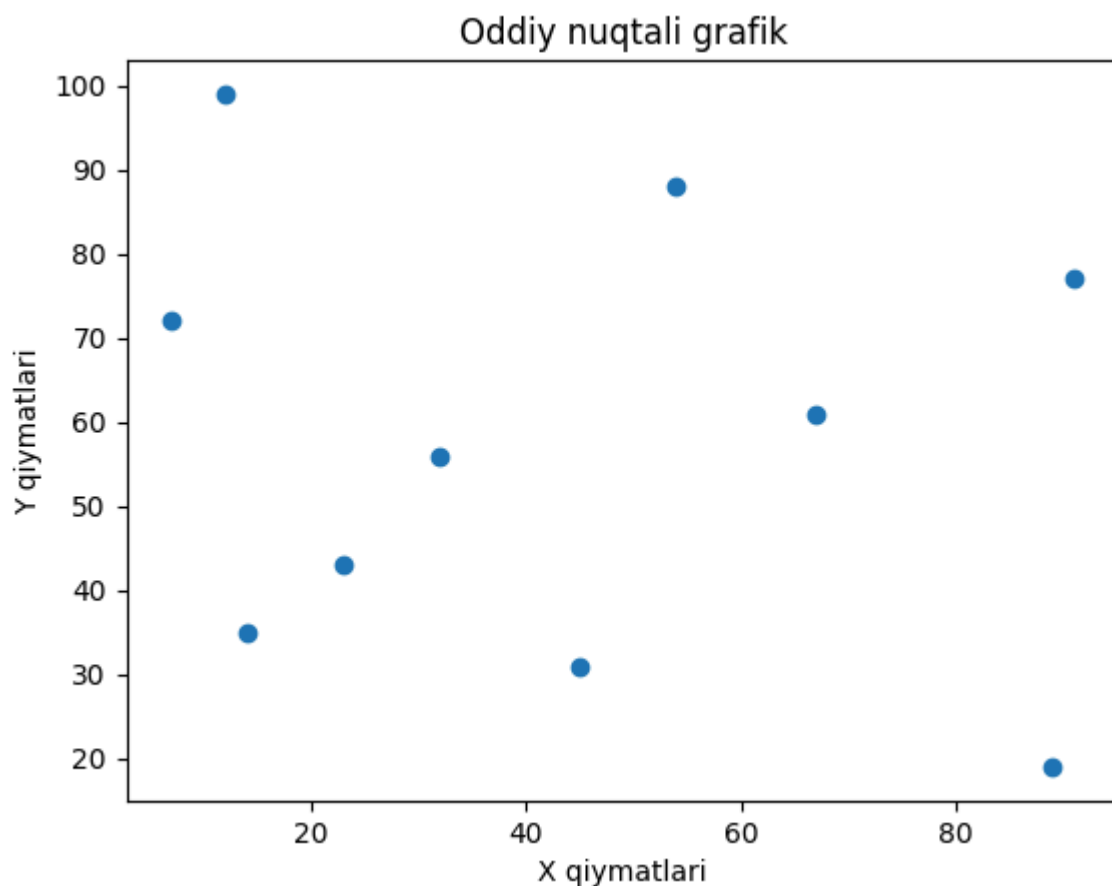
Masalan:

```
x = np.array([12, 45, 7, 32, 89, 54, 23, 67, 14, 91])
y = np.array([99, 31, 72, 56, 19, 88, 43, 61, 35, 77])
```

```
# Nuqtali grafik chizish
plt.scatter(x, y)

# Sarlavha va o'qlarga nom berish
plt.title("Oddiy nuqtali grafik")
plt.xlabel("X qiymatlari")
plt.ylabel("Y qiymatlari")

# Grafikni ekranda ko'rsatish
plt.show()
```



Tushuntirish: `plt.scatter(x, y)` ikkita o'zgaruvchi o'rtasidagi bog'liqlikni vizuallashtirish uchun 2D (ikki o'lchovli) tekislikda nuqtali grafik yaratadi. Grafik tushunarli bo'lishi va uning mazmunini ochib berish uchun unga sarlavha va o'q nomlari qo'shiladi.

Sintaksis

```
matplotlib.pyplot.scatter(x, y, s=None, c=None, marker=None, cmap=None, alpha=None,
edgecolors=None, label=None)
```

Parametrlar:

Parametr	Tavsifi (Description)
<code>x, y</code>	Chiziladigan ma'lumot nuqtalarining ketma-ketligi (massiv yoki ro'yxat).

Parametr	Tavsifi (Description)
<code>s</code>	Belgi (marker) o'lchami. U yagona raqam (skalyar) yoki har bir nuqta uchun alohida o'lchamlarni saqlovchi massiv bo'lishi mumkin.
<code>c</code>	Belgilarning rangi.
<code>marker</code>	Belgining shakli (masalan, <code>'o'</code> - dumaloq, <code>'*'</code> - yulduzcha, <code>'s'</code> - kvadrat).
<code>cmap</code>	Raqamli qiymatlarni ranglarga moslashtirish uchun foydalaniladigan ranglar xaritasi (Colormap).
<code>alpha</code>	Shaffoflik darajasi (<code>0</code> = to'liq shaffof/ko'rinmas, <code>1</code> = to'liq bo'yalgan/shaffof emas).
<code>edgecolors</code>	Belgi chetlarining (hoshiyasining) rangi.
<code>label</code>	Ma'lumotlar to'plami uchun shartli belgilar izohi (<code>legend</code>) nomlanishi.

Qaytaradigan qiymati (Returns): Bu funksiya nuqtali grafik nuqtalarini ifodalovchi `PathCollection` obyektini qaytaradi. Ushbu obyekt grafikni keyinchalik yanada moslashtirish (dizaynini o'zgartirish) yoki uni dinamik ravishda yangilash uchun ishlatilishi mumkin.

Nuqtali grafiklarga misollar (Examples)

1-misol: Ikkita guruhni taqqoslash

Ushbu misolda biz ikkita turli guruhning bo'yi va vaznini, har bir guruh uchun turli xil ranglardan foydalangan holda o'zaro taqqoslaymiz.

```
# 1-guruh ma'lumotlari (Bo'yi va vazni)
x1 = np.array([160, 165, 170, 175, 180, 185, 190, 195, 200, 205])
y1 = np.array([55, 58, 60, 62, 64, 66, 68, 70, 72, 74])

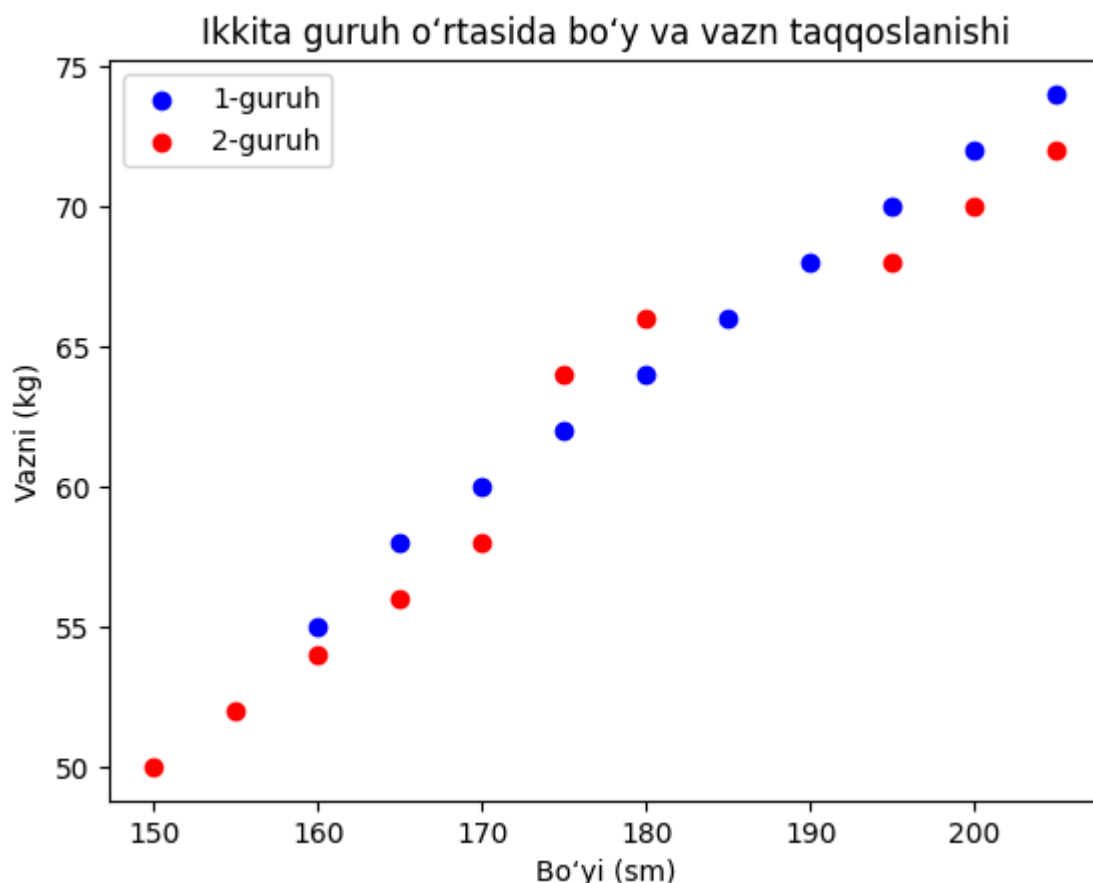
# 2-guruh ma'lumotlari
x2 = np.array([150, 155, 160, 165, 170, 175, 180, 195, 200, 205])
y2 = np.array([50, 52, 54, 56, 58, 64, 66, 68, 70, 72])

# Grafik chizish
plt.scatter(x1, y1, color='blue', label='1-guruh')
```

```
plt.scatter(x2, y2, color='red', label='2-guruh')

# O'qlarga nom va sarlavha berish
plt.xlabel('Bo'yi (sm)')
plt.ylabel('Vazni (kg)')
plt.title('Ikkita guruh o'rtasida bo'y va vazn taqqoslanishi')

# Izoh (Legend) qo'shish
plt.legend()
plt.show()
```



Izoh: Biz ikki guruhning bo'yi va vazni ma'lumotlari uchun `x1`, `y1` va `x2`, `y2` NumPy massivlarini aniqlaymiz. `plt.scatter()` funksiyasidan foydalanib, 1-guruh ko'k rangda va 2-guruh qizil rangda, har biri o'z nomlari (label) bilan chiziladi. Tushunarli bo'lishi uchun X va Y o'qlari mos ravishda "Bo'yi (sm)" va "Vazni (kg)" deb nomlangan.

2-misol: Har xil o'lcham va rangdagi nuqtalar

Ushbu misol har bir nuqta uchun turli xil belgi o'lchamlari va ranglaridan foydalangan holda nuqtali grafikni qanday moslashtirish (customize) mumkinligini ko'rsatadi. Shuningdek, nuqtalarning shaffofligi va hoshiya ranglari ham sozlanadi. Odatda bunday grafiklar **pufakchali diagramma** (Bubble chart) deb ham ataladi.

```
# X va Y koordinatalari
x = np.array([3, 12, 9, 20, 5, 18, 22, 11, 27, 16])
```

```

y = np.array([95, 55, 63, 77, 89, 50, 41, 70, 58, 83])

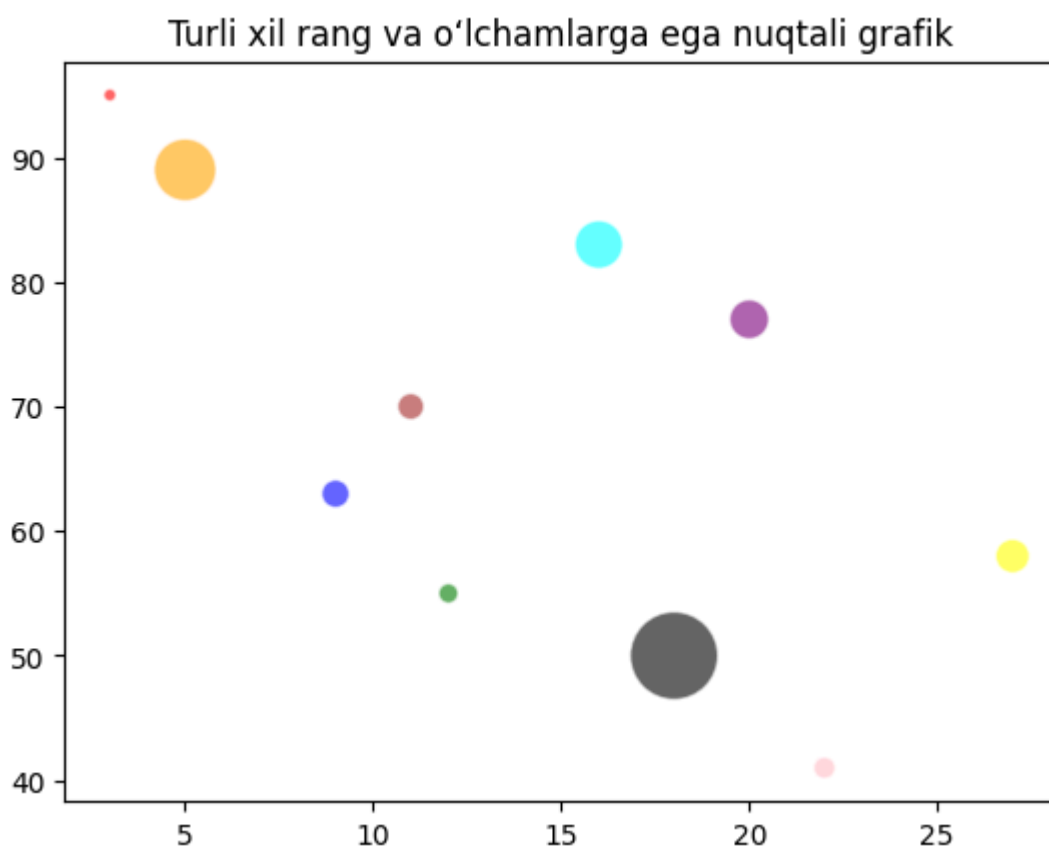
# a massivi har bir nuqtaning o'lchamini belgilaydi
a = [20, 50, 100, 200, 500, 1000, 60, 90, 150, 300]

# b ro'yxati har bir nuqtaning rangini belgilaydi
b = ['red', 'green', 'blue', 'purple', 'orange', 'black', 'pink', 'brown', 'brown', 'red']

# Nuqtali grafik chizish (s=o'lcham, c=rang, alpha=shaffoflik, edgecolors=ho
plt.scatter(x, y, s=a, c=b, alpha=0.6, edgecolors='w', linewidths=1)

# Sarlavha qo'shish
plt.title("Turli xil rang va o'lchamlarga ega nuqtali grafik")
plt.show()

```



Izoh: `x` va `y` NumPy massivlari nuqta koordinatalarini o'rnatadi, `a` massivi belgi o'lchamlarini aniqlaydi va `b` ro'yxati ularga ranglarni biriktiradi. `plt.scatter()` funksiyasi `alpha=0.6` yordamida nuqtalarni qisman shaffof qilib, shuningdek, oq rangli hoshiyalar (`edgecolors='w'`) va chiziqli qalinligi (`linewidths=1`) bilan chizadi. Grafikni ekranda ko'rsatishdan oldin unga sarlavha qo'shiladi.

3-misol: Pufakchali diagramma (Bubble Plot)

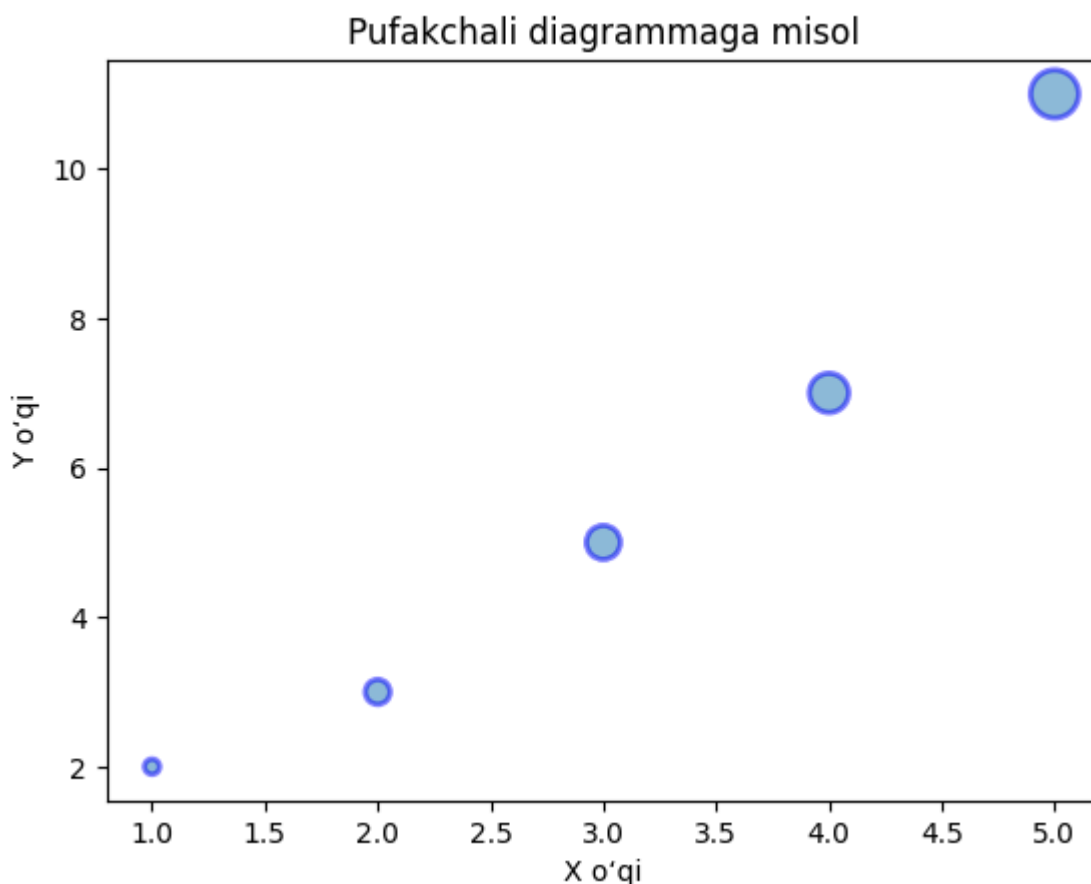
Ushbu misol har bir nuqtaning (pufakchani) o'lchami o'zgaruvchining miqdorini ifodalovchi pufakchali diagrammani qanday yaratishni ko'rsatadi. Bu yerda

hoshiya rangi va shaffoflik (alpha) parametrlaridan ham foydalanilgan.

```
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]
sizes = [30, 80, 150, 200, 300] # Pufakchalar o'lchami

# Grafik chizish
plt.scatter(x, y, s=sizes, alpha=0.5, edgecolors='blue', linewidths=2)

plt.title("Pufakchali diagrammaga misol")
plt.xlabel("X o'qi")
plt.ylabel("Y o'qi")
plt.show()
```



Izoh: `x` va `y` ro'yxatlari nuqta koordinatalarini belgilaydi, `sizes` esa belgi (pufakcha) o'lchamlarini o'rnatadi. `plt.scatter()` funksiyasi pufakchalarni 50% shaffoflik (`alpha=0.5`), ko'k rangli hoshiyalar va qalinligi 2 bo'lgan hoshiya chiziqlari bilan chizadi. Grafikni ekranda ko'rsatishdan oldin o'q nomlari va sarlavha qo'shilgan.

4-misol: Ranglar xaritasi (Colormap) va ranglar shkalasi (Colorbar) bilan nuqtali grafik

Ushbu misolda biz ma'lumotlar qiymatlarini ranglar xaritasi (colormap) yordamida ranglarga moslashtiramiz va yoniga ranglar shkalasini (colorbar) qo'shamiz. Bu uchinchi o'zgaruvchini rang jadalligi orqali vizuallashtirishga katta yordam beradi.

```

# 50 dan 150 gacha bo'lgan 100 ta tasodifiy sonlar
x = np.random.randint(50, 150, 100)
y = np.random.randint(50, 150, 100)

# Ranglarni moslashtirish uchun tasodifiy haqiqiy sonlar (float)
colors = np.random.rand(100)

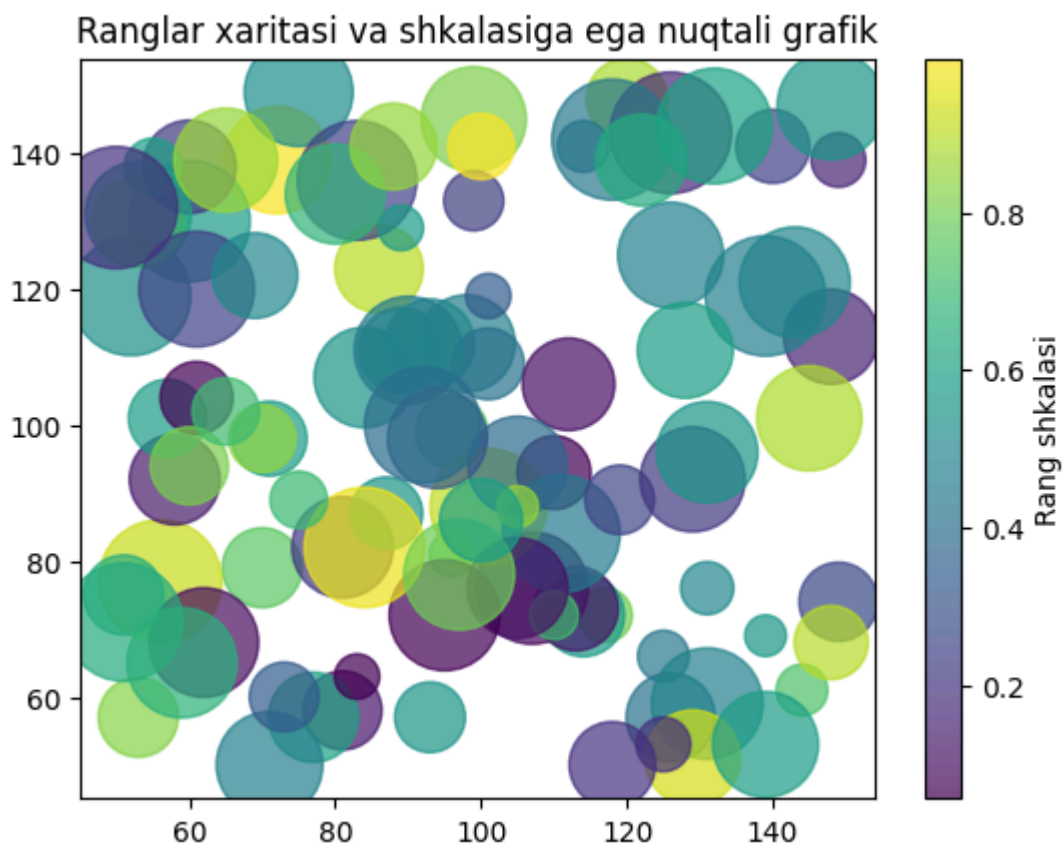
# 10 dan 100 gacha bo'lgan sonlarni 20 ga ko'paytirib o'lchamlar massivini y
sizes = 20 * np.random.randint(10, 100, 100)

# Nuqtali grafik chizish (cmap - ranglar xaritasi 'viridis')
plt.scatter(x, y, c=colors, s=sizes, cmap='viridis', alpha=0.7)

# Yon tomonga ranglar shkalasini qo'shish
plt.colorbar(label='Rang shkalasi')

plt.title("Ranglar xaritasi va shkalasiga ega nuqtali grafik")
plt.show()

```



Izoh: Tasodifiy `x` va `y` massivlari 100 ta nuqta koordinatalarini o'rnatadi, bunda ranglar `'viridis'` xaritasi yordamida moslashtirilgan va o'lchamlar turlicha. `plt.scatter()` funksiyasi ularni 0.7 shaffoflik bilan chizadi va `plt.colorbar()` funksiyasi esa grafik yoniga qaysi rang qanday qiymatga mos kelishini ko'rsatuvchi ranglar izohini (shkalani) qo'shadi.

5-misol: Turli xil belgi shakllaridan foydalanish

Ushbu so‘nggi misol `marker` parametridan foydalanib belgi (marker) shaklini qanday o‘zgartirishni ko‘rsatadi. Bu yerda to‘q qizil (magenta) rangdagi uchburchak belgilari ishlatilgan.

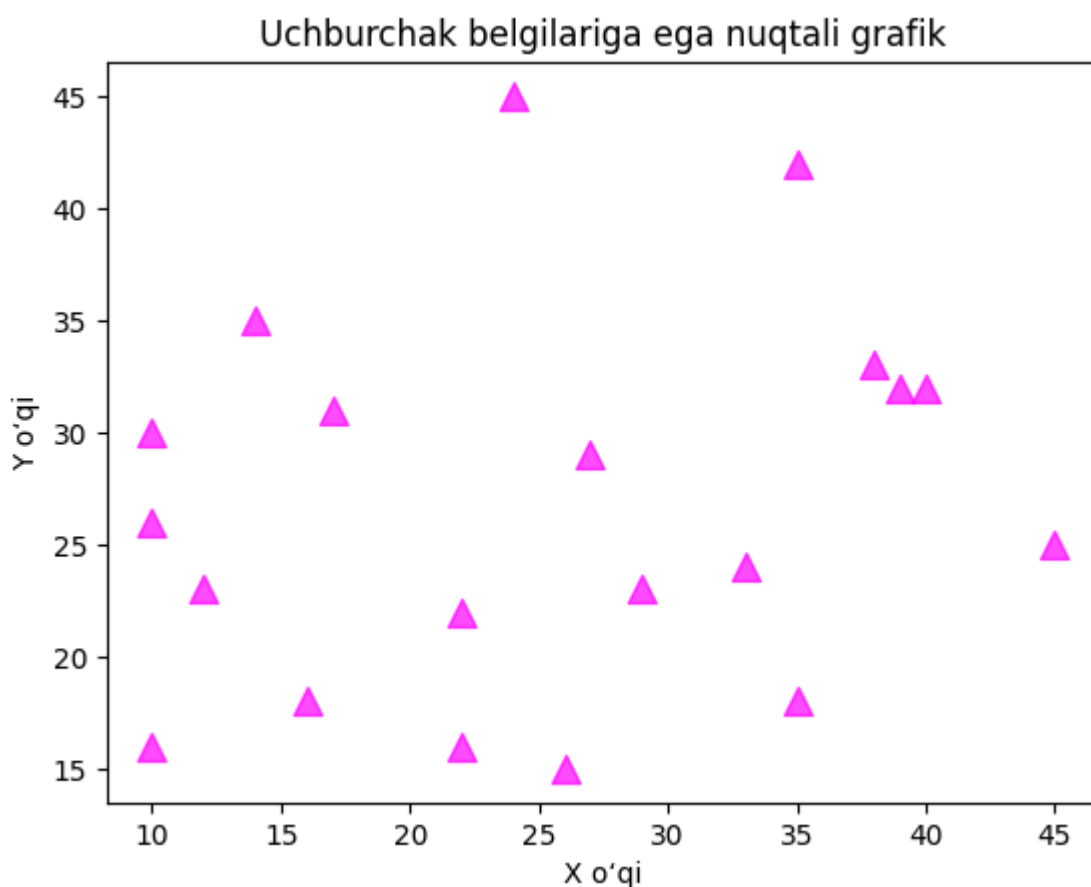
(Eslatma: Kod to‘liq va ishlaydigan bo‘lishi uchun `x` va `y` o‘zgaruvchilariga tasodifiy qiymatlar qo‘shildi).

```
# Tasodifiy ma'lumotlarni shakllantirish
x = np.random.randint(10, 50, 20)
y = np.random.randint(10, 50, 20)

# marker='^' orqali nuqtalarni uchburchak shaklida chizish
plt.scatter(x, y, marker='^', color='magenta', s=100, alpha=0.7)

# Sarlavha va o'qlarga nom berish
plt.title("Uchburchak belgilariga ega nuqtali grafik")
plt.xlabel("X o'qi")
plt.ylabel("Y o'qi")

# Grafikni ko'rsatish
plt.show()
```



Izoh: Ushbu kod o‘lchami 100 va shaffofligi 0.7 bo‘lgan, to‘q qizil (magenta) rangdagi uchburchak belgilari (`'^'`) yordamida nuqtalarni chizadi. Grafikni ekranda ko‘rsatishdan oldin unga sarlavha qo‘shilgan.

2.5. Gistogrammalar (Histograms)

Gistogrammalar (Histograms)

Gistogrammalar **ma'lumotlarni** vizuallashtirishda eng asosiy vositalardan biridir. Ular har bir qiymat yoki qiymatlar oralig'i qanchalik tez-tez uchrashini ko'rsatib, ma'lumotlar taqsimotining grafik ifodasini taqdim etadi. Gistogrammalar uzluksiz raqamli ma'lumotlarni, masalan, o'lchovlar, datchik ko'rsatkichlari yoki tajriba natijalarini tahlil qilish uchun ayniqsa foydalidir.

Gistogramma ustunli diagrammaning o'ziga xos turi bo'lib, unda:

- **X o'qi** ma'lumotlarning oraliqlarini (**binlar** deb ataladi) ifodalaydi.
- **Y o'qi** har bir oraliq (bin) ichidagi qiymatlar chastotasini (qancha marta takrorlanishini) ifodalaydi.

Oddiy ustunli diagrammalardan farqli o'laroq, gistogrammalar ma'lumotlar taqsimotini samarali umumlashtirish uchun ma'lumotlarni uzluksiz oraliqlarga (binlarga) guruhlaydi.

Matplotlib yordamida gistogramma yaratish (Creating a Matplotlib Histogram)

1. Ma'lumotlar oralig'ini ketma-ket, bir-biriga ustma-ust tushmaydigan oraliqlarga (binlarga) bo'ling.
2. Har bir oraliqqa (binga) nechta qiymat tushishini hisoblang.
3. Gistogrammani chizish uchun `matplotlib.pyplot.hist()` funksiyasidan foydalaning.

Quyidagi jadvalda `matplotlib.pyplot.hist()` funksiyasi qabul qiladigan asosiy parametrlar keltirilgan:

Parametr (Parameter)	Vazifasi va tavsifi
<code>x</code>	Raqamli ma'lumotlar massivi yoki ketma-ketligi.
<code>bins</code>	Oraliqlar soni (butun son) yoki aniq oraliqlar chegaralari (massiv).
<code>density</code>	Agar <code>True</code> bo'lsa, chastota (takrorlanish soni) o'rniga ehtimollikni ko'rsatish uchun gistogrammani normallashtiradi.

Parametr (Parameter)	Vazifasi va tavsifi
<code>range</code>	Oraliqlarning pastki va yuqori chegaralarini belgilovchi juftlik (tuple).
<code>histtype</code>	Gistogramma turi: <code>bar</code> , <code>barstacked</code> , <code>step</code> , <code>stepfilled</code> . Standart qiymat: <code>bar</code> .
<code>align</code>	Ustunlarni tekislash: <code>left</code> (chapga), <code>right</code> (o'ngga), <code>mid</code> (markazga).
<code>weights</code>	Har bir ma'lumot nuqtasi uchun vaznlar massivi.
<code>bottom</code>	Oraliqlar uchun asosiy chiziq (pastki chegara).
<code>rwidth</code>	Ustunlarning nisbiy kengligi (0 dan 1 gacha).
<code>color</code>	Ustunlarning rangi. Bitta rang yoki ranglar ketma-ketligi bo'lishi mumkin.
<code>label</code>	Shartli belgilar izohi (legend) uchun nom.
<code>log</code>	Agar <code>True</code> bo'lsa, Y o'qida logarifmik shkaladan (logarithmic scale) foydalanadi.

Python'da Matplotlib yordamida gistogramma chizish

Bu yerda biz Python'dagi Matplotlib kutubxonasida gistogramma chizishning turli xil usullarini ko'rib chiqamiz:

- Oddiy gistogramma
- Zichlik grafigi (Density Plot) bilan moslashtirilgan gistogramma
- Suv belgisi (Watermark) bilan moslashtirilgan gistogramma
- Quyi grafiklar (Subplots) yordamida bir nechta gistogrammalar
- Ustma-ust joylashgan (Stacked) gistogramma
- 2D gistogramma (Hexbin Plot)

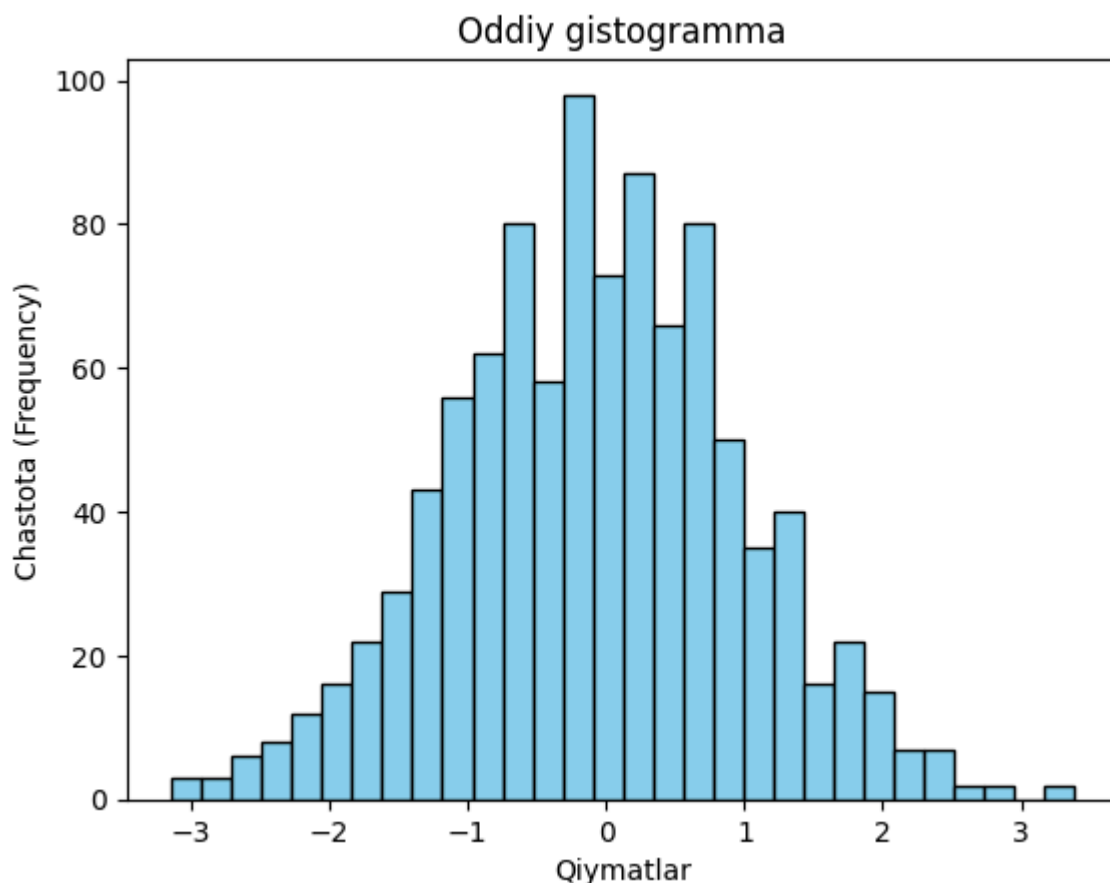
1. Oddiy gistogramma (Basic Histogram)

```
# Gistogramma uchun tasodifiy ma'lumotlarni yaratish
data = np.random.randn(1000)
```

```
# Oddiy gistogrammani chizish (30 ta oraliq, och ko'k ustunlar va qora hoshi)
plt.hist(data, bins=30, color='skyblue', edgecolor='black')

# O'qlarga nom berish va sarlavha qo'shish
plt.xlabel('Qiymatlar')
plt.ylabel('Chastota (Frequency)')
plt.title('Oddiy gistogramma')

# Grafikni ekranda ko'rsatish
plt.show()
```



Izoh:

- Standart normal taqsimotdan 1000 ta tasodifiy sonlarni yaratadi.
- 30 ta oraliq (`bins=30`), och ko'k rangli ustunlar va qora hoshiyalar yordamida gistogramma chizadi.
- X va Y o'qlariga nomlar hamda sarlavha qo'shadi.
- Gistogrammani ekranda ko'rsatadi.
- Bu ma'lumotlar taqsimotini vizuallashtirishning eng oddiy usulidir.

2. Zichlik grafigi (Density Plot) bilan moslashtirilgan gistogramma

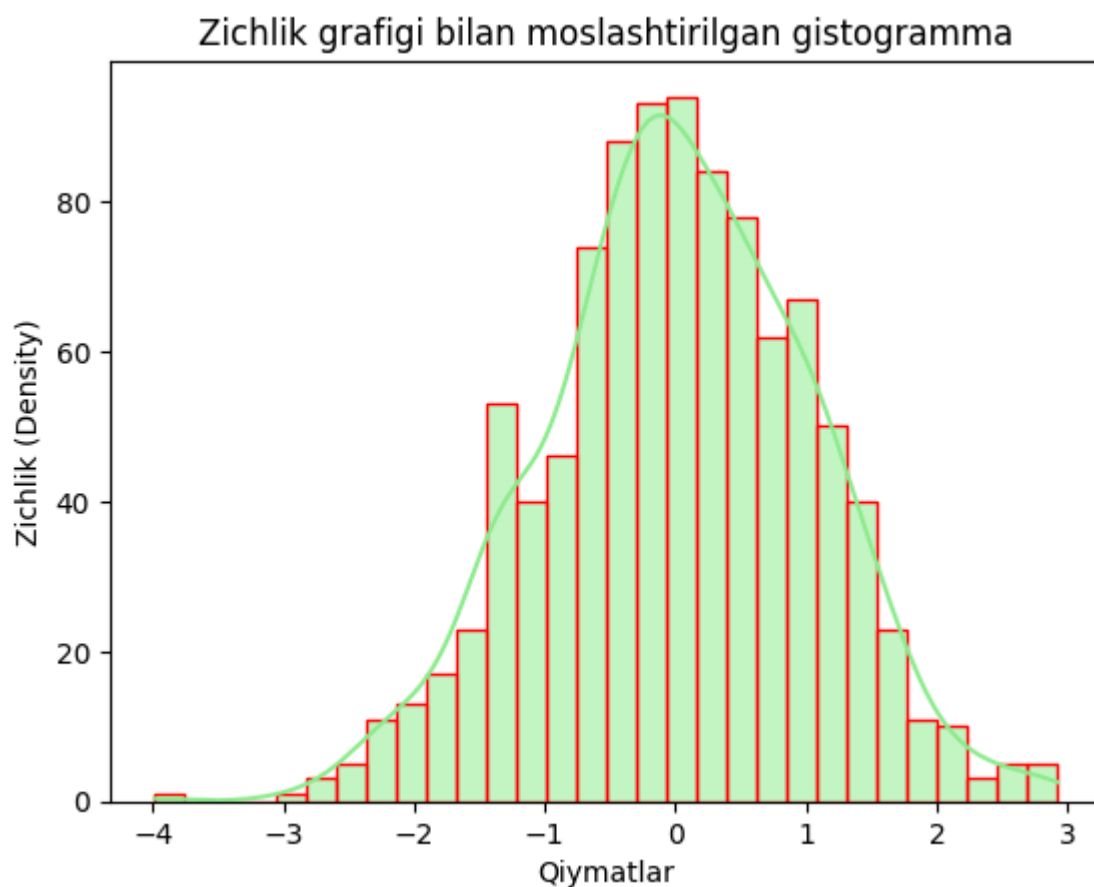
```
import seaborn as sns
```

```
# Gistogramma uchun tasodifiy ma'lumotlarni yaratish
data = np.random.randn(1000)

# Zichlik grafigi (KDE) bilan moslashtirilgan gistogramma yaratish
sns.histplot(data, bins=30, kde=True, color='lightgreen', edgecolor='red')

# O'qlarga nom berish va sarlavha qo'shish
plt.xlabel('Qiymatlar')
plt.ylabel('Zichlik (Density)')
plt.title('Zichlik grafigi bilan moslashtirilgan gistogramma')

# Grafikni ekranda ko'rsatish
plt.show()
```



Izoh:

- 1000 ta tasodifiy sonlarni yaratadi.
- Seaborn kutubxonasidan foydalanib, 30 ta oraliqqa (binlarga) ega gistogramma va silliq zichlik egri chizig'ini (KDE - Kernel Density Estimate) chizadi.
- Ustunlarni och yashil rangga bo'yaydi va qizil hoshiyalar qo'shadi.
- O'qlarga nomlar va sarlavha qo'shadi.
- Ma'lumotlar taqsimoti va zichligini ko'rsatuvchi grafikni ekranda namoyish etadi.

3. Suv belgisi (Watermark) bilan moslashtirilgan gistogramma

```
from matplotlib import colors

# Ma'lumotlar to'plamini yaratish
np.random.seed(23685752) # Natijalar takrorlanishi (bir xil chiqishi) uchun
N_points = 10000
n_bins = 20

# Taqsimot yaratish
x = np.random.randn(N_points)
y = 0.8 ** x + np.random.randn(N_points) + 25
legend = ['Taqsimot']

# Oyna (figure) va o'qlarni yaratish
fig, axs = plt.subplots(1, 1, figsize=(10, 7), tight_layout=True)

# O'qlarning ramka chiziqlarini (spines) olib tashlash
for s in ['top', 'bottom', 'left', 'right']:
    axs.spines[s].set_visible(False)

# X va Y o'qlaridagi chiziqlarni (ticks) olib tashlash
axs.xaxis.set_ticks_position('none')
axs.yaxis.set_ticks_position('none')

# O'qlar va belgilashlar orasiga bo'sh joy (padding) qo'shish
axs.xaxis.set_tick_params(pad=5)
axs.yaxis.set_tick_params(pad=10)

# X va Y o'qlarida to'r chiziqlarini (gridlines) qo'shish
axs.grid(visible=True, color='grey', linestyle='-.', linewidth=0.5, alpha=0.5)

# Matnli suv belgisini (watermark) qo'shish
fig.text(0.9, 0.15, 'Jeeteshgavande30',
        fontsize=12,
        color='red',
        ha='right',
        va='bottom',
        alpha=0.7)

# Gistogramma yaratish
N, bins, patches = axs.hist(x, bins=n_bins)

# Ranglar gradientini o'rnatish
fracs = ((N ** (1 / 5)) / N.max())
norm = colors.Normalize(fracs.min(), fracs.max())

for thisfrac, thispatch in zip(fracs, patches):
    color = plt.cm.viridis(norm(thisfrac))
```



```
thispatch.set_facecolor(color)
```

```
# Qo'shimcha xususiyatlar (nomlar, izoh va sarlavha) qo'shish
```

```
plt.xlabel("X o'qi")
```

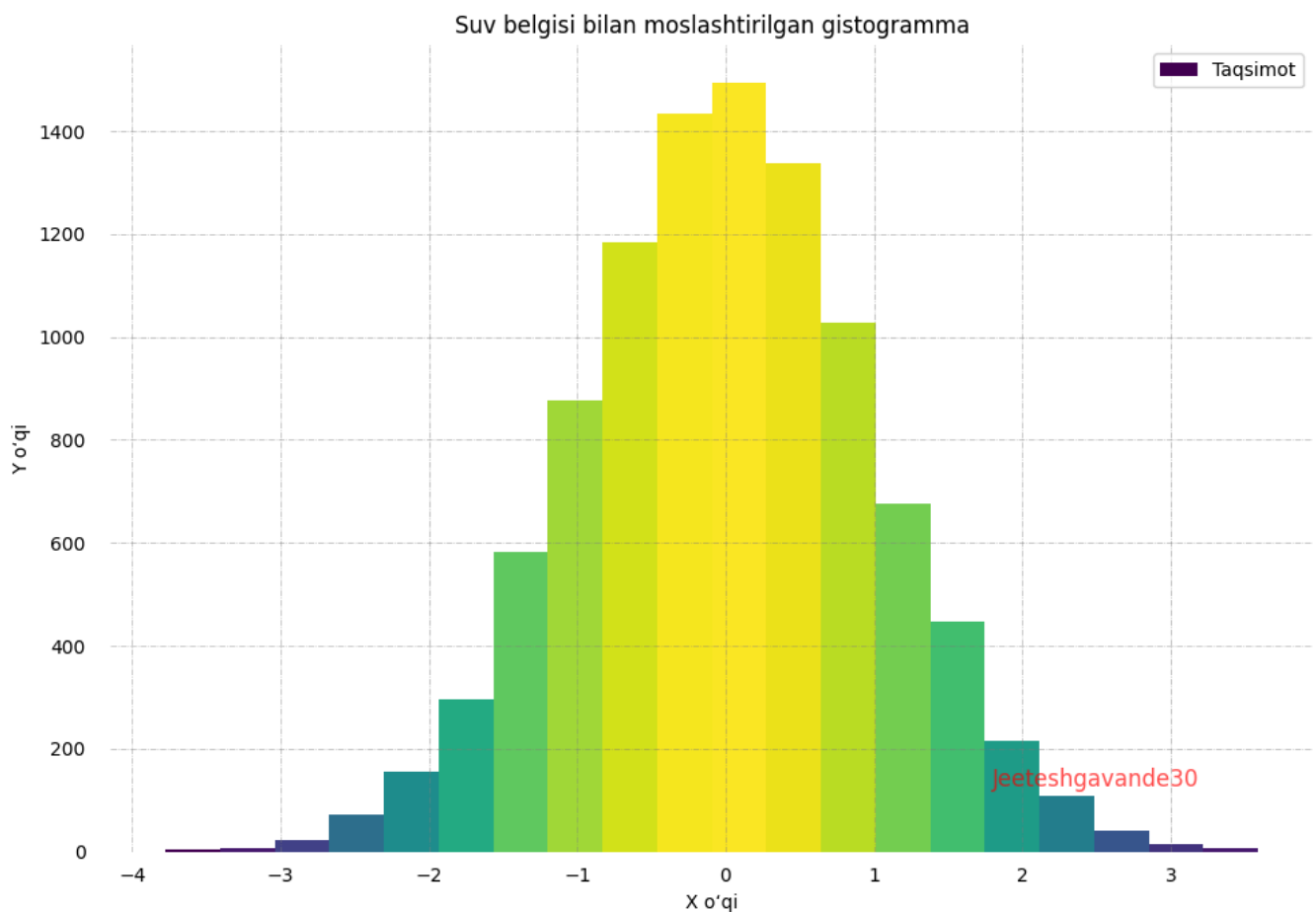
```
plt.ylabel("Y o'qi")
```

```
plt.legend(legend)
```

```
plt.title('Suv belgisi bilan moslashtirilgan gistogramma')
```

```
# Grafikni ko'rsatish
```

```
plt.show()
```



Izoh:

- Natijalarni qayta ishlab chiqarish (reproducibility) uchun tasodifiy sonlar generatori (seed) o'rnatiladi.
- Standart normal taqsimotdan 10,000 ta tasodifiy son (`x`) yaratadi.
- 20 ta oraliq (`bins`) yordamida gistogramma chizadi va ustunlarga rang gradientini qo'llaydi.
- O'qlarning chetki chiziqlarini va belgilash chiziqchalarini olib tashlaydi, to'r chiziqlarini va suv belgisini qo'shadi.
- O'q nomlarini, shartli belgilar izohini va sarlavhani qo'shadi.
- Vizual jihatdan yaxshilangan va moslashtirilgan gistogrammani ekranda namoyish etadi.

4. Quyi grafiklar (Subplots) yordamida bir nechta gistogrammalar

Ushbu usul bitta umumiy oynada (figure) bir nechta grafiklarni yonma-yon yoki ostma-ust joylashtirish imkonini beradi. Bu turli xil ma'lumotlar to'plamlarini o'zaro solishtirish uchun juda qulay.

```
# Bir nechta gistogrammalar uchun tasodifiy ma'lumotlarni yaratish
data1 = np.random.randn(1000)
data2 = np.random.normal(loc=3, scale=1, size=1000)

# Quyi grafiklar (subplots) yordamida oynani yaratish (1 qator, 2 ta ustun)
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))

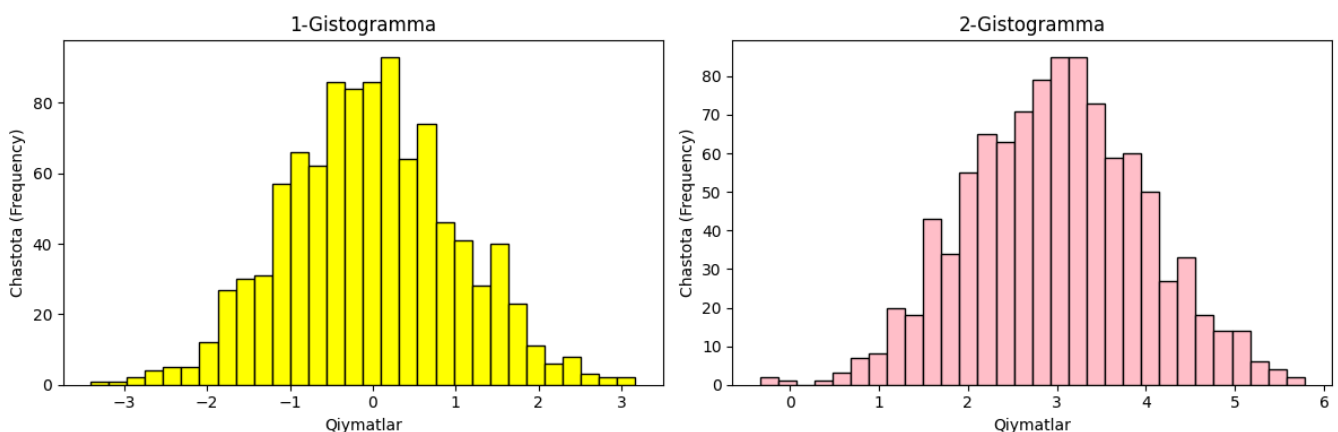
# Birinchi grafik (Sariq rangda)
axes[0].hist(data1, bins=30, color='yellow', edgecolor='black')
axes[0].set_title('1-Gistogramma')

# Ikkinchi grafik (Pushti rangda)
axes[1].hist(data2, bins=30, color='pink', edgecolor='black')
axes[1].set_title('2-Gistogramma')

# Har ikkala grafikka o'q nomlarini qo'shish
for ax in axes:
    ax.set_xlabel('Qiymatlar')
    ax.set_ylabel('Chastota (Frequency)')

# Joylashuvni yaxshiroq moslashtirish (yozuvlar ustma-ust tushib qolmasligi)
plt.tight_layout()

# Grafikni ekranda ko'rsatish
plt.show()
```



Izoh:

- Har biri 1000 ta tasodifiy sondan iborat bo'lgan ikkita ma'lumotlar to'plamini (data1 va data2) yaratadi.

- `subplots` yordamida yonma-yon joylashgan gistogrammalarni hosil qiladi (`nrows=1` , `ncols=2`).
- `data1` ma'lumotlarini sariq, `data2` ma'lumotlarini esa pushti rangda 30 ta oraliq (`bins=30`) yordamida chizadi.
- O'qlarga nomlar va sarlavhalar qo'shadi, joylashuvni vizual jihatdan to'g'rilaydi (`tight_layout`) va grafikni namoyish etadi.

5. Ustma-ust joylashgan (Stacked) gistogramma

Ushbu usul ikkita yoki undan ortiq ma'lumotlar to'plamini bitta ustun ichida ustma-ust yig'ib ko'rsatish uchun ishlatiladi. Bu umumiy taqsimotni va uning tarkibiy qismlarini bir vaqtning o'zida ko'rish imkonini beradi.

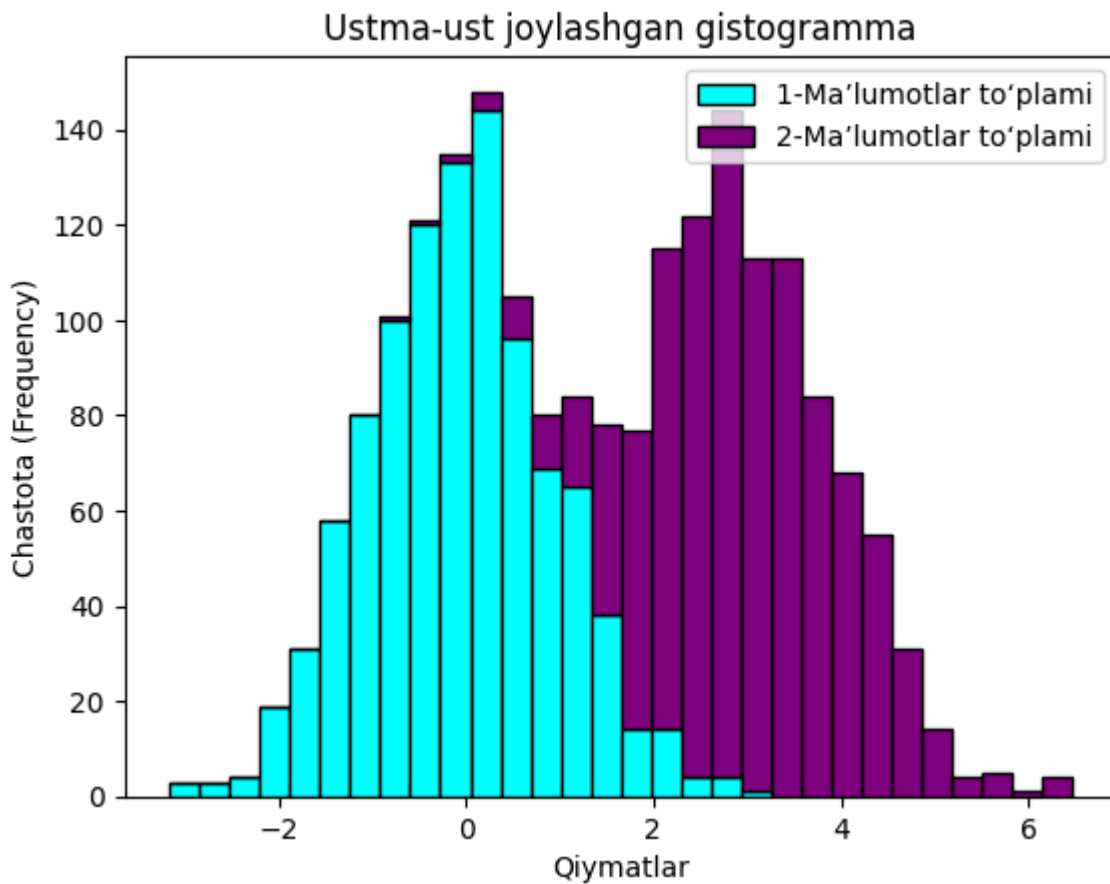
```
# Ustma-ust joylashgan gistogrammalar uchun tasodifiy ma'lumotlarni yaratish
data1 = np.random.randn(1000)
data2 = np.random.normal(loc=3, scale=1, size=1000)

# Ustma-ust joylashgan gistogrammani yaratish (stacked=True)
plt.hist([data1, data2], bins=30, stacked=True, color=['cyan', 'purple'], ec

# O'qlarga nom berish va sarlavha qo'shish
plt.xlabel('Qiymatlar')
plt.ylabel('Chastota (Frequency)')
plt.title('Ustma-ust joylashgan gistogramma')

# Shartli belgilar izohini (Legend) qo'shish
plt.legend(['1-Ma'lumotlar to'plami', '2-Ma'lumotlar to'plami'])

# Grafikni ekranda ko'rsatish
plt.show()
```



Izoh:

- Har biri 1000 ta tasodifiy sondan iborat bo'lgan ikkita ma'lumotlar to'plamini yaratadi.
- `stacked=True` parametridan foydalanib, ikkala ma'lumotlar to'plami birlashtirilgan holda ustma-ust joylashgan gistogrammani hosil qiladi.
- Qora hoshiyaga ega feruza (cyan) va binafsha (purple) rangli ustunlardan foydalanadi.
- O'qlarga nomlar, sarlavha va shartli belgilar izohini qo'shadi.
- Birlashtirilgan ma'lumotlar taqsimotini ko'rsatuvchi gistogrammani ekranda namoyish etadi.

6. 2D Gistogramma (Hexbin Plot)

2D gistogrammalar (yoki Hexbin Plot — Oltiburchakli grafiklar) ma'lumotlarning X va Y o'qlari kesishmasida qanchalik zich joylashganini vizual tarzda ko'rsatadi. U nuqtali grafiklarning (scatter plot) muqobili bo'lib, ayniqsa ma'lumotlar hajmi juda ko'p bo'lganida nuqtalar ustma-ust tushib ketishining oldini olish va zichlikni yaxshiroq tushunish uchun ishlatiladi.

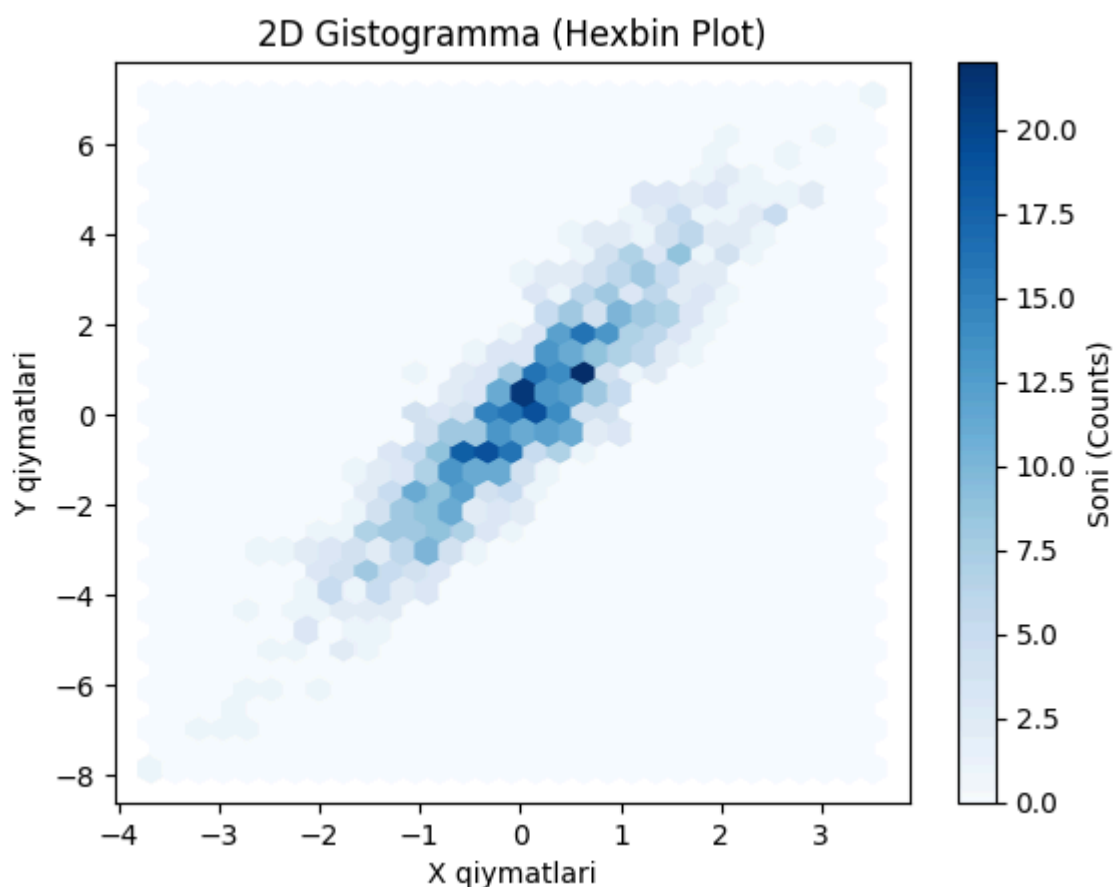
```
# Hexbin grafigi uchun tasodifiy 2D ma'lumotlarni yaratish
x = np.random.randn(1000)
y = 2 * x + np.random.normal(size=1000)
```

```
# 2D gistogramma (hexbin plot) yaratish
# gridsize - oltiburchaklar to'rining zichligi
plt.hexbin(x, y, gridsize=30, cmap='Blues')

# O'qlarga nom berish va sarlavha qo'shish
plt.xlabel('X qiymatlari')
plt.ylabel('Y qiymatlari')
plt.title('2D Gistogramma (Hexbin Plot)')

# Ranglar shkalasini (colorbar) qo'shish
plt.colorbar(label='Soni (Counts)')

# Grafikni ekranda ko'rsatish
plt.show()
```



Izoh:

- Har biri 1000 ta tasodifiy sondan iborat bo'lgan ikkita to'plamni (`x` va `y`) yaratadi.
- Oltiburchaklar yordamida ma'lumotlar zichligini ko'rsatuvchi 2D gistogramma (hexbin plot) chizadi.
- Zichlik darajasini ifodalash uchun ko'k ranglar xaritasidan (`cmap='Blues'`) foydalanadi.

- O‘q nomlari, sarlavha va zichlikni qanday talqin qilish kerakligini ko‘rsatuvchi ranglar shkalasini (colorbar) qo‘shadi.
- `x` va `y` o‘rtasidagi bog‘liqlik va zichlikni ko‘rsatuvchi 2D gistogrammani ekranda namoyish etadi.

2.6. Doiraviy diagrammalar (Pie Charts)

Doiraviy diagramma — bu faqat bitta ma’lumotlar seriyasini ko‘rsata oladigan doira shaklidagi statistik grafikdir. Diagrammaning umumiy maydoni berilgan ma’lumotlarning umumiy miqdorini (100% ni) tashkil etadi. Python'da doiraviy diagrammalar ma’lumotlar taqsimotini ko‘rsatishdagi soddaligi va samaradorligi tufayli biznes taqdimotlar, hisobotlar va boshqaruv panellarida (dashboards) keng qo‘llaniladi. Ushbu qismda biz Python'da ma’lumotlarni vizuallashtirish uchun eng ko‘p ishlatiladigan kutubxonalardan biri bo‘lgan Matplotlib yordamida doiraviy diagramma yaratishni ko‘rib chiqamiz.

Reja:

1. Nima uchun doiraviy diagrammalardan foydalanamiz?
2. Doiraviy diagrammaning asosiy tuzilishi
3. Matplotlib yordamida doiraviy diagramma chizish
4. Doiraviy diagrammalarni moslashtirish
5. Python'da ichma-ich joylashgan (Nested) doiraviy diagramma yaratish
6. 3D doiraviy diagrammalar yaratish

Nima uchun doiraviy diagrammalardan foydalanamiz?

Doiraviy diagrammalar qismlarni butunga nisbatan taqqoslashni osonlashtiradigan vizual ifodani taqdim etadi. Ular quyidagi holatlarda ayniqsa foydalidir:

- **Nisbiy nisbatlar yoki foizlarni ko‘rsatishda.**
- **Toifalangan (categorical) ma’lumotlarni umumlashtirishda.**
- **Toifalar o‘rtasidagi muhim farqlarni ta’kidlashda.**

Biroq, doiraviy diagrammalar qanchalik foydali bo‘lmasin, ularning o‘ziga xos cheklovlari ham bor. Agar toifalar soni juda ko‘p bo‘lsa, ular tartibsiz ko‘rinishga kelishi yoki to‘g‘ri ishlab chiqilmasa, noto‘g‘ri talqin qilinishiga olib kelishi mumkin. Shunga qaramay, Matplotlib yordamida yaxshi ishlangan doiraviy diagramma ma’lumotlaringiz taqdimotini sezilarli darajada yaxshilashi mumkin.

Doiraviy diagrammaning asosiy tuzilishi

Doiraviy diagramma turli toifalarni ifodalovchi bo‘laklardan (kesmalardan) iborat. Har bir bo‘lakning o‘lchami u ifodalayotgan miqdorga proporsional (mutanosib) bo‘ladi. Matplotlib'da doiraviy diagramma yaratishda quyidagi tarkibiy qismlar muhim ahamiyatga ega:

- **Ma'lumotlar (Data):** Har bir toifa uchun qiymatlar yoki miqdorlar (hisoblar).
- **Nomlar (Labels):** Har bir toifaning nomlari, ular bo‘laklar yonida ko‘rsatiladi.
- **Ranglar (Colors):** Ixtiyoriy, lekin ranglardan bo‘laklarni bir-biridan samarali va vizual tarzda ajratish uchun foydalanish mumkin.

Matplotlib API o‘zining `pyplot` modulida massivdagi ma'lumotlarni ifodalovchi doiraviy diagramma yaratadigan `pie()` funksiyasiga ega. Keling, uning sintaksisi va parametrlari bilan tanishib chiqamiz.

Sintaksis:

```
matplotlib.pyplot.pie(data, explode=None, labels=None, colors=None, autopct=None, shadow=False)
```

Parametrlar (Parameters):

- **data** : Chizilishi kerak bo‘lgan ma'lumotlar qiymatlari massivini ifodalaydi. Har bir bo‘lakning umumiy maydondagi ulushi `data/sum(data)` formulasi orqali aniqlanadi.
- **labels** : Har bir bo‘lakka (wedge) nom berish uchun ishlatiladigan satrlar (strings) ro‘yxati.
- **colors** : Bo‘laklarga rang berish uchun ishlatiladigan atribut (ranglar ro‘yxati).
- **autopct** : Bo‘laklarni ularning raqamli (odatda foiz) qiymati bilan belgilash uchun ishlatiladigan formatlash satri (masalan, `'%1.1f%%'` — qiymatni yuzdan bir aniqlikdagi foizda ko‘rsatadi).
- **shadow** : Bo‘laklarning ostida soya yaratish uchun ishlatiladi (standart holatda `False`, agar `True` qilinsa 3D effektiga o‘xshash soya paydo bo‘ladi).

Matplotlib yordamida doiraviy diagramma chizish

Keling, Matplotlib'dagi `pie()` funksiyasidan foydalanib oddiy doiraviy diagramma yaratamiz. Ushbu funksiya toifalangan (categorical) ma'lumotlar taqsimotini vizuallashtirishning kuchli va juda oson usulidir.

```

# Kutubxonalarni chaqirish
from matplotlib import pyplot as plt
import numpy as np

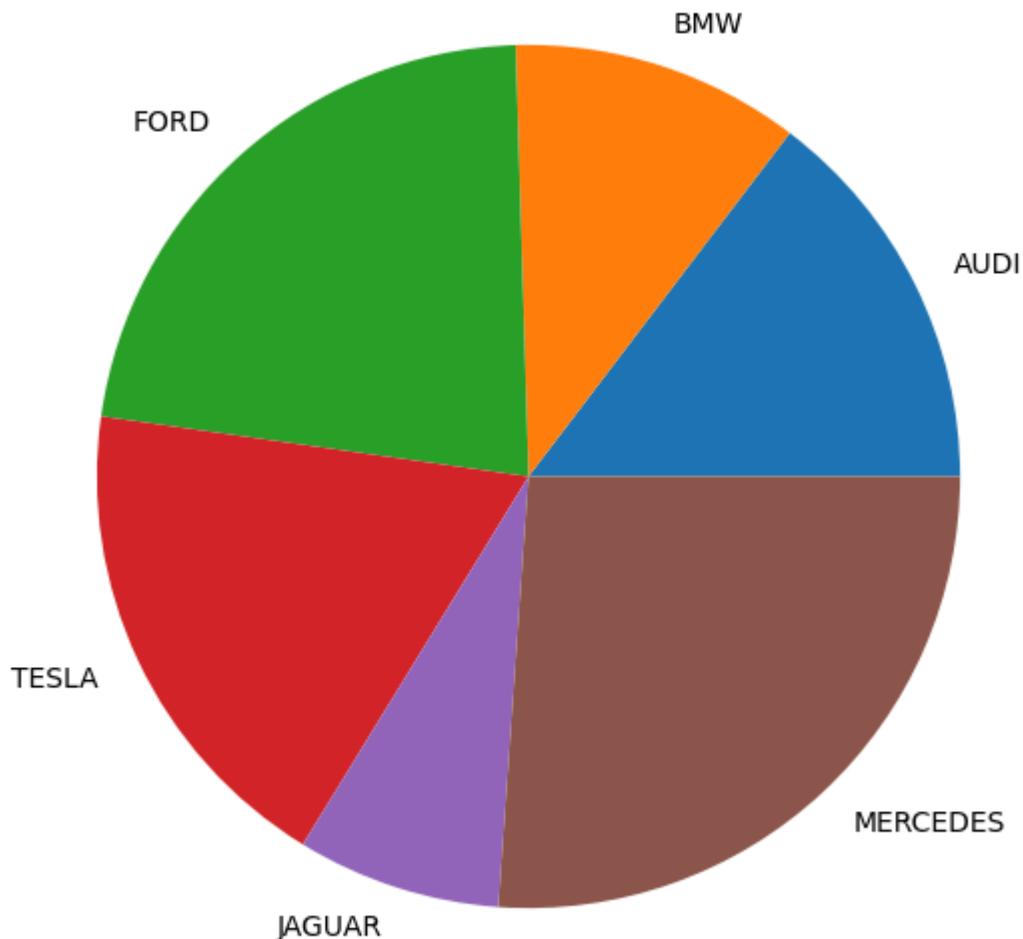
# Ma'lumotlar to'plamini yaratish (avtomobil rusumlari va ularning soni)
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR', 'MERCEDES']
data = [23, 17, 35, 29, 12, 41]

# Grafik oynasini yaratish (o'lchami 10x7 dyuym)
fig = plt.figure(figsize=(10, 7))

# Doiraviy diagrammani chizish (labels orqali har bir bo'lakka nom berish)
plt.pie(data, labels=cars)

# Grafikni ekranda ko'rsatish
plt.show()

```



Izoh:

- Biz birinchi navbatda kerakli kutubxonalarni (`matplotlib.pyplot` va `numpy`) import qilib olamiz.

- Doiraviy diagrammada ko'rsatiladigan avtomobil rusumlari ro'yxati (`cars`) va ularga mos keluvchi qiymatlar massivini (`data`) yaratamiz.
- `plt.figure(figsize=(10, 7))` orqali o'lchami 10x7 bo'lgan bo'sh oyna (figure) tayyorlaymiz.
- `plt.pie(data, labels=cars)` funksiyasi yordamida berilgan raqamlar qanday proporsiyada ekanligini hisoblab, doiraviy diagramma chizamiz va har bir kesmaga avtomobil nomini biriktiramiz.
- Eng oxirida `plt.show()` orqali natijani ekranda namoyish etamiz.

Doiraviy diagrammalarni moslashtirish (Customizing Pie Charts)

Matplotlib'dagi doiraviy diagrammalarning asoslari bilan tanishib chiqqaningizdan so'ng, ularni ehtiyojlaringizga moslashtirishni boshlashingiz mumkin. Doiraviy diagrammani bir nechta xususiyatlar (parametrlar) asosida sozlash mumkin:

- **startangle (Boshlanish burchagi):** Bu parametr doiraviy diagrammani ko'rsatilgan gradusga qarab x-o'qi bo'ylab soat miliga qarshi burish imkonini beradi. Ushbu burchakni o'zgartirish orqali siz birinchi bo'lakning boshlanish holatini o'zgartirishingiz mumkin, bu esa diagrammaning umumiy ko'rinishini yaxshilashi mumkin.
- **shadow (Soya):** Ushbu mantiqiy (boolean) atribut doira ostida soya effektini qo'shadi. Uni `True` qilib belgilash orqali siz diagrammani bo'rttirib, unga uch o'lchamli (3D) ko'rinish berishingiz mumkin.
- **wedgeprops (Bo'lak xususiyatlari):** Bu parametr doiraviy diagrammadagi har bir bo'lakning xususiyatlarini moslashtirish uchun Python lug'atini (dictionary) qabul qiladi. Siz `linewidth` (chiziq qalinligi) va `edgecolor` (hoshiya rangi) kabi turli xususiyatlarni belgilashingiz mumkin.
- **frame (Ramka):** Agar `True` qilib belgilansa, diagramma atrofida ramka chiziladi.
- **autopct :** Bu parametr bo'laklarda foizlar qanday ko'rsatilishini nazorat qiladi.
- **explode (Ajratish):** Ushbu parametr diagrammaning bir yoki bir nechta bo'laklarini markazdan ajratib (tashqariga surib) ko'rsatishga xizmat qiladi.
- Shuningdek, qo'shimcha ravishda `legend()` (shartli belgilar) va `set_title()` (sarlavha) funksiyalari orqali grafikni yanada tushunarli

qilish mumkin.

Keling, bularning barchasini bitta kompleks misolda ko'ramiz:

```
# Ma'lumotlar to'plami
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR', 'MERCEDES']
data = [23, 17, 35, 29, 12, 41]

# Ajratish parametrlari (Qaysi bo'lak qanchalik uzoqqa surilishi)
# Bu yerda FORD (0.2) va TESLA (0.3) hammadan ko'proq ajratib ko'rsatilgan
explode = (0.1, 0.0, 0.2, 0.3, 0.0, 0.0)

# Ranglar parametrlari
colors = ("orange", "cyan", "brown", "grey", "indigo", "beige")

# Bo'lak xususiyatlari (qalinligi 1 va hoshiyasi yashil)
wp = {'linewidth': 1, 'edgecolor': "green"}

# autopct uchun funksiya yaratish (foiz va aniq miqdorni chiqarish uchun)
def func(pct, allvalues):
    absolute = int(pct / 100. * np.sum(allvalues))
    # {:.1f}% - yuzdan bir aniqlikdagi foiz, \n - yangi qator, {:d} - butun
    return "{:.1f}%\n({:d} ta)".format(pct, absolute)

# Grafik oyna va o'qlarni yaratish
fig, ax = plt.subplots(figsize=(10, 7))

# Doiraviy diagrammani chizish (barcha parametrlarni qo'shish bilan)
wedges, texts, autotexts = ax.pie(data,
                                   autopct= lambda pct: func(pct, data),
                                   explode=explode,
                                   labels=cars,
                                   shadow=True,
                                   colors=colors,
                                   startangle=90, # 90 gradusdan chizishni boshlab
                                   wedgeprops=wp,
                                   textprops=dict(color="magenta")) # yozuvlar

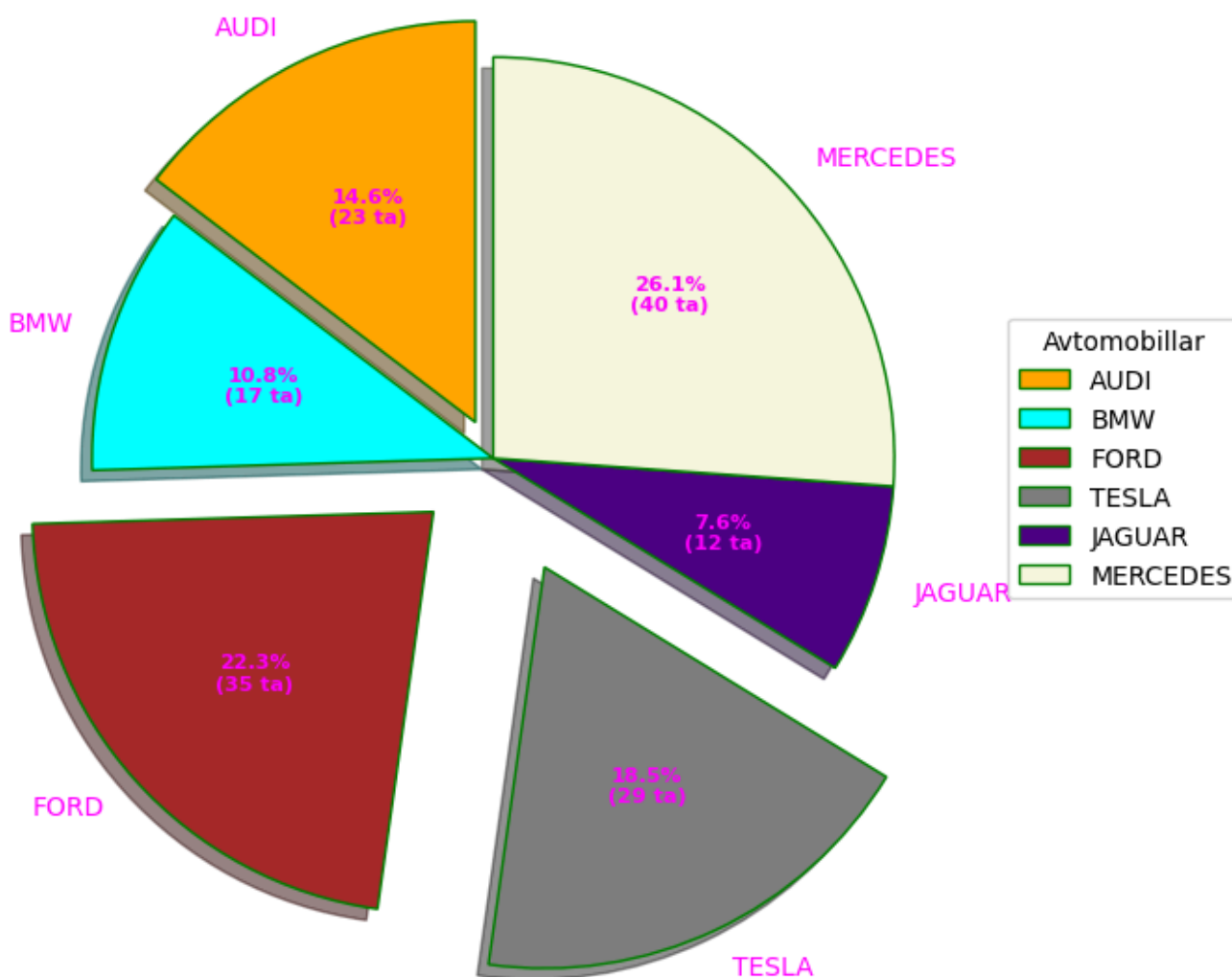
# Shartli belgilar (legend) qo'shish
ax.legend(wedges, cars,
          title="Avtomobillar",
          loc="center left",
          bbox_to_anchor=(1, 0, 0.5, 1)) # O'ng tomonga joylashtirish

# Ichki yozuvlarning (autotexts) hajmi va qalinligini sozlash
plt.setp(autotexts, size=8, weight="bold")

ax.set_title("Moslashtirilgan doiraviy diagramma")
```

```
# Grafikni ko'rsatish  
plt.show()
```

Moslashtirilgan doiraviy diagramma



Ushbu kod orqali siz Matplotlib yordamida juda ham boy vizual ko'rinishga ega, professional doiraviy diagramma chizishingiz mumkin.

Matplotlib'dagi `plt.pie()` funksiyasi imkoniyatlaridan foydalanib, biz ma'lumotlarni samarali yetkazishga yordam beradigan ma'lumotlarga boy va vizual jihatdan jozibali doiraviy diagrammalar yaratishimiz mumkin. Ma'lumotlarni manfaatdor tomonlarga taqdim etasizmi yoki hisobotlaringiz uchun ko'rgazmali qurollar yaratasizmi, Python'da doiraviy diagrammalar chizish san'atini egallash juda qimmatli ko'nikma hisoblanadi.

Python'da ichma-ich joylashgan (Nested) doiraviy diagramma yaratish

Ichma-ich joylashgan doiraviy diagramma iyerarxik (pog'onali) ma'lumotlarni ifodalashning samarali usuli bo'lib, bitta umumiy ko'rinishda bir nechta toifalar va kichik toifalarni (subcategories) vizuallashtirish imkonini beradi.

Matplotlib'da turli radiusli bir nechta doiraviy diagrammalarni ustma-ust tushirish (yoki qutb o'qlari yordamida) orqali bunday diagrammani yaratishingiz mumkin.

Quyida Python'da ichma-ich joylashgan doiraviy diagrammani qanday yaratish bo'yicha misol keltirilgan:

```
# Ma'lumotlar to'plamini yaratish
size = 6
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR', 'MERCEDES']
data = np.array([[23, 16], [17, 23],
                 [35, 11], [29, 33],
                 [12, 27], [41, 42]])

# Ma'lumotlarni 2 * pi (360 gradus) ga moslashtirish (normallashtirish)
norm = data / np.sum(data) * 2 * np.pi

# Ustun (bo'lak) chetlarining ordinatalarini olish
left = np.cumsum(np.append(0, norm.flatten()[:-1])).reshape(data.shape)

# Ranglar shkalasini yaratish (Colormap)
cmap = plt.get_cmap("tab20c")
outer_colors = cmap(np.arange(6) * 4) # Tashqi halqa ranglari
inner_colors = cmap(np.array([1, 2, 5, 6, 9,
                              10, 12, 13, 15,
                              17, 18, 20])) # Ichki halqa ranglari

# Grafik oyna va qutb (polar) o'qlarini yaratish
fig, ax = plt.subplots(figsize=(10, 7),
                          subplot_kw=dict(polar=True))

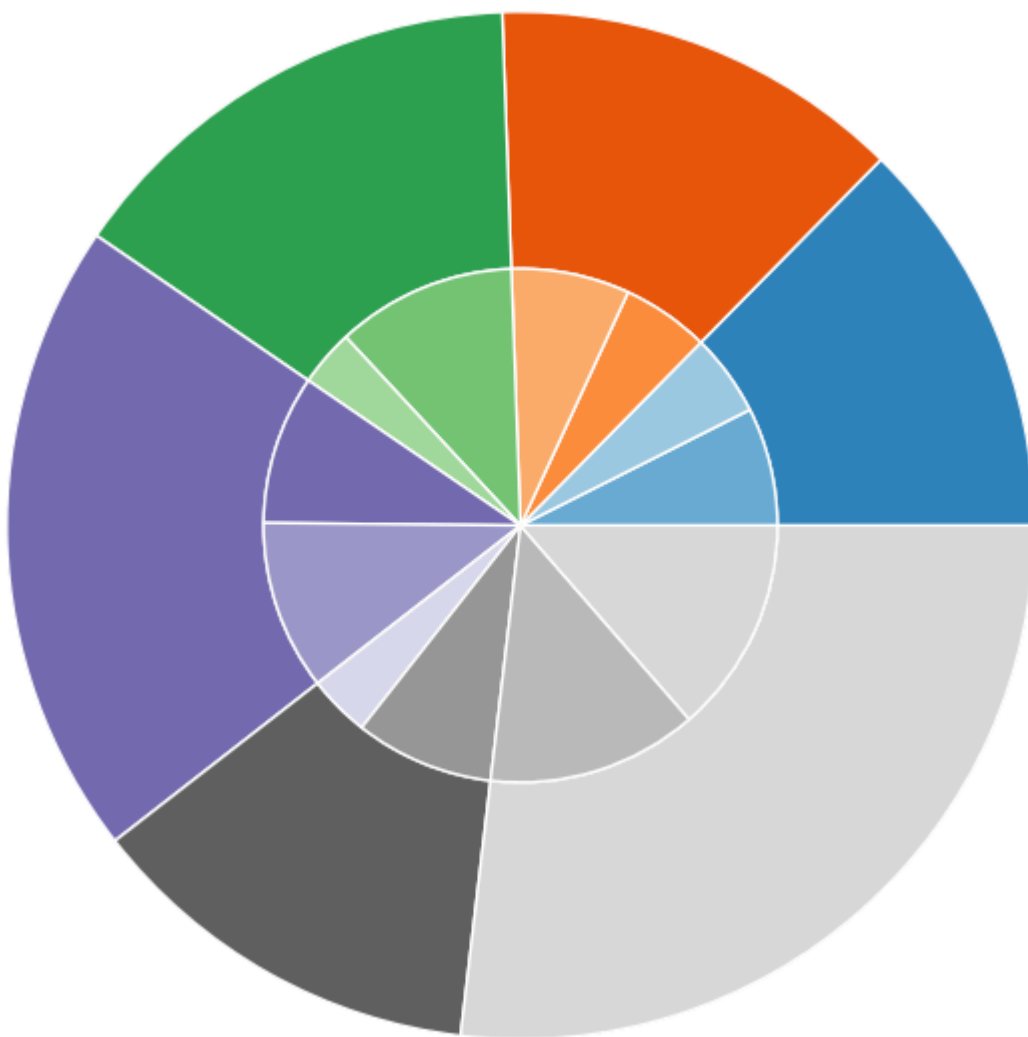
# Tashqi halqani chizish
ax.bar(x=left[:, 0],
       width=norm.sum(axis=1),
       bottom=1-size,
       height=size,
       color=outer_colors,
       edgecolor='w', # Oq rangli hoshiya
       linewidth=1,
       align="edge")

# Ichki halqani chizish
ax.bar(x=left.flatten(),
       width=norm.flatten(),
       bottom=1-2 * size,
       height=size,
       color=inner_colors,
       edgecolor='w',
       linewidth=1,
       align="edge")
```

```
# Sarlavha qo'shish va o'qlarni (koordinata chiziqlarini) yashirish
ax.set(title="Ichma-ich joylashgan doiraviy diagramma")
ax.set_axis_off()

# Grafikni ko'rsatish
plt.show()
```

Ichma-ich joylashgan doiraviy diagramma



Izoh: Ushbu kod ichma-ich doiraviy diagramma yaratish uchun standart `pie()` funksiyasidan emas, balki qutb koordinatalar tizimidagi (`polar=True`) ustunli grafik chizuvchi `bar()` funksiyasidan foydalanadi. Bu ancha ilg'or va juda moslashuvchan usul hisoblanadi:

- `data` massivi har bir toifa (avtomobil rusumi) uchun 2 ta ostki (ichki) qiymatni o'z ichiga oladi.
- `ax.bar` yordamida avval umumiy yig'indilar orqali **tashqi halqa**, so'ngra individual qiymatlar orqali markazga yaqinroq bo'lgan **ichki halqa** chiziladi.

- Halqalarning radiusi (`bottom` va `height` parametrlari orqali) hamda ularning aylanadagi yoyi kengligi (`x` va `width` orqali) matematik tarzda 360 gradusga (`2 * np.pi`) mutanosib taqsimlab chiqilgan. Oqibatda iyerarxik ma'lumotlarni bitta vizual ko'rinishda juda chiroyli tarzda ifodalashga erishilgan.

Tashqi doiraviy diagramma asosiy toifalarni, ichki doiraviy diagramma esa o'sha asosiy toifalardan biriga tegishli kichik toifalarni (subcategories) ifodalaydi. Ushbu tuzilish mutanosiblik (proporsiya) ichidagi mutanosiblikni ko'rsatish uchun ayniqsa foydali bo'lib, tomoshabinlarga ma'lumotlar ichidagi munosabatlarni tezda anglab olishga yordam beradi.

Markaziy aylana (Center Circle): Tashqi va ichki doiraviy diagrammalar o'rtasida toza vizual ajratishni ta'minlovchi "donut" (teshikko'lcha) effektini yaratish uchun qo'shiladi.

Python'dagi oddiy doiraviy diagrammalarda bo'lgani kabi, ichma-ich joylashgan diagrammalarning umumiy estetikasini (ko'rinishini) yaxshilash uchun `startangle` , `shadow` , `autopct` va `wedgeprops` kabi turli parametrlarni sozlashingiz mumkin.

3D doiraviy diagrammalar yaratish

Matplotlib'da haqiqiy 3D doiraviy diagramma yaratish uchun quyidagi kod parchasidan foydalanishingiz mumkin. Shuni yodda tutingki, Matplotlib'da 3D doiraviy diagrammalar uchun to'g'ridan-to'g'ri (maxsus) funksiya yo'q, lekin biz uni 3D sirt (surface) grafigi orqali simulyatsiya qilishimiz yoki 2D doiraviy diagrammalarga qo'shimcha ishlov berish orqali unga erishishimiz mumkin.

(Matplotlib doiraviy diagrammalarga "3D-dek" ko'rinish berish uchun asosan `shadow=True` yoki turli radiusli qatlamlardan foydalanishni taklif etadi. Agar sizga haqiqiy, aylantirib ko'rish mumkin bo'lgan 3D pie chart kerak bo'lsa, odatda `plotly` kabi boshqa kutubxonalar ishlatiladi. Ammo Matplotlib doirasida bu simulyatsiya bilan amalga oshiriladi).

Xulosa

Ushbu maqolada biz Matplotlib kutubxonasidan foydalanib Python'da doiraviy diagrammalar (pie charts) yaratish va ularni moslashtirish asoslarini o'rganib chiqdik. Matplotlib'da oddiy doiraviy diagramma qurishdan tortib, murakkabroq ma'lumotlar to'plamlarini 2D va 3D doiraviy diagrammalar yordamida vizuallashtirishgacha, ko'rgazmali qurollarimizning samaradorligini oshirishi mumkin bo'lgan turli jihatlarni qamrab oldik.

Python'dagi `plt.pie()` funksiyasidan foydalanib, biz toifalangan (categorical) ma'lumotlarni qanday qilib aniq-ravshan taqdim etishni va shu orqali olingan xulosalarni (insights) manfaatdor tomonlarga yetkazishni qanchalik osonlashtirish mumkinligini bilib oldik.